TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



NGUYỄN DƯƠNG HIẾU

THIẾT KẾ MẠNG WIFI MESH KẾT NỐI DRONE VÀ ROBOT

ĐỒ ÁN CHUYÊN NGÀNH

KỸ THUẬT ĐIỆN TỬ – VIỄN THÔNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



NGUYỄN DƯƠNG HIẾU

MSSV: 42300151

THIẾT KẾ MẠNG WIFI MESH KẾT NỐI DRONE VÀ ROBOT

ĐỒ ÁN CHUYÊN NGÀNH

KỸ THUẬT ĐIỆN TỬ – VIỄN THÔNG

Người hướng dẫn
PGS.TS. Đặng Ngọc Minh Đức

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Tôi xin chân thành cảm ơn PGS.TS. Đặng Ngọc Minh Đức đã tận tình hướng dẫn, góp ý và định hướng cho tôi trong suốt quá trình thực hiện đồ án. Tôi cũng xin cảm ơn quý thầy cô Khoa Điện – Điện tử, Trường Đại học Tôn Đức Thắng đã truyền đạt những kiến thức nền tảng về điện tử, viễn thông và mạng máy tính, giúp tôi có cơ sở để hoàn thành đề tài này. Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ tôi trong thời gian qua.

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Tác giả

Nguyễn Dương Hiếu

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của PGS.TS. Đặng Ngọc Minh Đức. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Đồ án chuyên ngành còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Đồ án chuyên ngành của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Tác giả

(ký tên và ghi rõ họ tên)

Nguyễn Dương Hiếu

NHIỆM VỤ ĐỒ ÁN CHUYÊN NGÀNH

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

LỊCH TRÌNH LÀM ĐỒ ÁN CHUYÊN NGÀNH

Họ tên sinh viên: NGUYỄN DƯƠNG HIẾU
Lớp: 23040202 MSSV: 42300151
Tên đề tài: Thiết kế mạng WiFi Mesh kết nối Drone và Robot

Tuần/Ngày	**Khối lượng**		**GVHD ký**
	Đã thực hiện	**Tiếp tục thực hiện**	
01/09 – 10/09/2026	Nghiên cứu lý thuyết mạng mesh, painlessMesh, ESP-WIFI-MESH; viết firmware Root/Leaf (phân mảnh chunk 1024 B, ghép ảnh); đặc tả giao thức UART; xây dựng middleware Node.js (kiểm thử parser 7/7); chốt danh sách linh kiện.	Nhận linh kiện; bring-up board Root; chạy thử middleware với phần cứng thật.	
12/09 – 13/09/2026	Bring-up board Root (driver CP210x, sửa lỗi firmware đòi PSRAM); Root gửi telemetry qua UART 921600; chạy middleware với Root thật, giảm mức log debug của painlessMesh.	Bring-up các node Leaf ESP32-CAM.	
14/09 – 16/09/2026	Bring-up 3 Leaf ESP32-CAM OV3660; sửa lỗi quét mesh (chuyển sang fork Alteriom/painlessMesh), tắt kết nối STA của Root; chẩn đoán tranh chấp RF và lỗi nguồn USB; soạn quy trình hàn nguồn pin.	Thử nghiệm 2 Leaf đồng thời; hàn nguồn pin.	

Tuần/Ngày	Khối lượng		GVHD ký
	Đã thực hiện	**Tiếp tục thực hiện**	
17/09 – 18/09/2026	Thử nghiệm 2 Leaf (tỉ lệ hủy khung ~82 %); xác định nguyên nhân do Root chỉ có 1 vùng đệm ghép; vẽ sơ đồ khối, lưu đồ; lập báo cáo tiến độ 18/09.	Sửa firmware: 4 vùng đệm ghép, gửi sendSingle về Root.	
26/09 – 27/09/2026	Firmware Root ghép 4 khung song song, cơ chế supersede, beacon #ROOT; Leaf học Root và gửi sendSingle; viết lại Dashboard với tab so sánh 1/2/3 Leaf.	Thử nghiệm 3 Leaf chạy pin.	
28/09 – 29/09/2026	Hàn nguồn pin cho 3 Leaf; thử nghiệm 3 Leaf (phát hiện toàn mạng ngừng truyền dữ liệu, giảm tải chu kỳ 600 ms); chốt dùng painlessMesh; báo cáo tiến độ giữa kỳ.	Bổ sung latency, topology, RSSI và các cơ chế ổn định.	
30/09/2026	Thêm mốc thời gian chụp vào header (16 byte), latency, cây topology, RSSI; watchdog Leaf, theo dõi node im lặng, bảo vệ heap, ghi UART không chặn, staggering.	Đo theo số hop, đo latency, đo dòng tiêu thụ, đo self-healing lặp lại.	
Kiểm tra giữa kỳ	Đánh giá khối lượng hoàn thành .....% được tiếp tục/không tiếp tục thực hiện đồ án		
Nộp đồ án	Đã hoàn thành .....% đồ án được bảo vệ/không được bảo vệ đồ án		

THIẾT KẾ MẠNG WIFI MESH KẾT NỐI DRONE VÀ ROBOT

TÓM TẮT

Đồ án trình bày quá trình thiết kế, thi công và thử nghiệm một mạng WiFi Mesh dùng vi điều khiển ESP32 để truyền ảnh JPEG từ các node mô phỏng drone và robot về một trạm giám sát mà không cần bộ định tuyến, Internet hay mạng di động. Hệ thống gồm một node gốc (Root) dùng board ESP32 DevKitC nối với máy tính và ba node lá (Leaf) dùng board ESP32-CAM với camera OV3660 chạy pin 18650, liên kết với nhau bằng thư viện painlessMesh theo cấu trúc cây tự tổ chức. Các node được đặt trên mặt đất hoặc cầm tay khi thử nghiệm, vì vậy đồ án là một mô hình truyền thông đa chặng theo hướng mạng ad-hoc cho thiết bị bay, chưa được thử nghiệm trên drone đang bay.

Mỗi ảnh JPEG chụp tại Leaf được chia thành các chunk mang tối đa 1024 byte dữ liệu kèm header 16 byte, gồm định danh node, định danh khung, chỉ số chunk và mốc thời gian chụp. Leaf học địa chỉ của Root qua bản tin beacon và gửi chunk thẳng tới Root; Root ghép song song ảnh của nhiều Leaf bằng bốn vùng đệm có bitmap và thời gian chờ, sau đó đóng khung nhị phân có mã CRC16 và gửi lên máy tính qua UART 921600 bps. Middleware Node.js chuyển dữ liệu qua Socket.IO tới Web Dashboard để hiển thị ảnh, FPS, thông lượng, tỉ lệ hủy khung, topology, RSSI, độ trễ và thời gian tự phục hồi.

Với một Leaf ở chu kỳ chụp 600 ms, hệ thống đạt 1,67 ± 0,40 khung/giây và không có khung bị hủy. Khi hai Leaf gửi đồng thời trong lúc Root chỉ có một vùng đệm ghép, tỉ lệ hủy khung lên tới khoảng 82 %; sau khi tăng lên bốn vùng đệm, FPS tổng ở chu kỳ 300 ms tăng gần tuyến tính lên khoảng 10 khung/giây với ba Leaf, nhưng toàn mạng ngừng truyền dữ liệu sau 6–26 giây do Root quá tải việc ghi UART. Khi tăng chu kỳ chụp lên 600 ms, ba Leaf chạy liên tục khoảng 2 phút với FPS tổng 4,85 ± 0,96 khung/giây. Đồ án cũng bổ sung watchdog, bảo vệ heap, ghi UART không chặn và gửi lệch pha giữa các Leaf; thời gian tự phục hồi đo được còn phân tán từ 885 đến 32528 ms nên chưa được dùng làm kết quả.

Từ khóa: WiFi Mesh, ESP32-CAM, painlessMesh, phân mảnh ảnh, tự phục hồi, Web Dashboard.

DESIGN OF A WIFI MESH NETWORK FOR CONNECTING DRONES AND ROBOTS

ABSTRACT

This project presents the design, implementation and testing of a WiFi mesh network built on ESP32 microcontrollers that relays JPEG images from nodes emulating drones and robots to a monitoring station without any router, Internet access or cellular network. The system consists of one ESP32 DevKitC Root node attached to a computer and three battery-powered ESP32-CAM OV3660 Leaf nodes, connected by the painlessMesh library as a self-organizing tree. The nodes were placed on the ground or carried by hand during the tests, so the work is a multi-hop communication model oriented towards flying ad-hoc networks rather than a deployment on flying drones.

Each JPEG frame captured at a Leaf is fragmented into chunks carrying up to 1024 bytes of image data and a 16-byte header with the node ID, frame ID, chunk index and capture timestamp. Each Leaf learns the Root address from a periodic beacon and sends its chunks directly to the Root, which reassembles frames from several Leaves in parallel using four buffers with bitmaps and timeouts, then wraps them in CRC16-protected binary frames sent to the computer over a 921600 bps UART link. A Node.js middleware forwards the data via Socket.IO to a Web Dashboard showing images, FPS, throughput, drop rate, topology, RSSI, latency and self-healing time.

With one Leaf at a 600 ms capture interval, the system reaches 1.67 ± 0.40 frames per second with no dropped frames. With two Leaves and a single reassembly buffer at the Root, about 82 % of the frames are dropped; with four buffers and a 300 ms interval, the aggregate frame rate grows almost linearly to about 10 frames per second with three Leaves, but the whole network stops delivering data after 6–26 seconds because the Root is overloaded by UART output. At a 600 ms interval, the three-Leaf network runs continuously for about 2 minutes at 4.85 ± 0.96 frames per second. Watchdogs, heap protection, non-blocking UART output and time-slotted transmission were also added, while the measured self-healing times still range from 885 to 32528 ms and are not yet used as a result.

Keywords: WiFi mesh, ESP32-CAM, painlessMesh, image fragmentation, self-healing, web dashboard.

MỤC LỤC

DANH MỤC HÌNH VẼ . . . . . **xiv**

DANH MỤC BẢNG BIỂU . . . . . **xv**

DANH MỤC CÁC CHỮ VIẾT TẮT . . . . . **xvi**

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI **1**
1.1 Đặt vấn đề 1
1.2 Mục tiêu của đề tài 2
1.2.1 Mục tiêu tổng quát 2
1.2.2 Mục tiêu cụ thể 2
1.3 Nhiệm vụ của đề tài 3
1.4 Đối tượng và phạm vi nghiên cứu 4
1.4.1 Đối tượng nghiên cứu 4
1.4.2 Phạm vi nghiên cứu 4
1.5 Phương pháp thực hiện 4
1.6 Bố cục báo cáo 5
1.7 Kết luận chương 5

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **6**
2.1 Mạng mesh không dây 6
2.1.1 Khái niệm và đặc điểm 6
2.1.2 So sánh với các mô hình mạng khác 7
2.2 Các giao thức định tuyến cho mạng ad-hoc 7
2.2.1 Phân loại giao thức định tuyến MANET 7
2.2.2 Giao thức B.A.T.M.A.N 8
2.2.3 Mạng ad-hoc cho thiết bị bay 9
2.3 ESP-WIFI-MESH và thư viện painlessMesh 11
2.3.1 ESP-WIFI-MESH 11
2.3.2 Thư viện painlessMesh 12
2.4 Hệ điều hành thời gian thực FreeRTOS trên ESP32 13

2.4.1 Tác vụ và các cơ chế giao tiếp . . . 13
2.4.2 Bộ lập lịch . . . 14
2.4.3 FreeRTOS trên ESP32 hai lõi . . . 14
2.4.4 Cơ chế watchdog . . . 15
2.4.5 Áp dụng trong đề tài . . . 15
2.5 Nền tảng phần cứng . . . 17
2.5.1 Module ESP32-CAM và camera OV3660 . . . 17
2.5.2 Board ESP32 DevKitC dùng cho Root . . . 17
2.6 Các chỉ số đánh giá hiệu năng mạng . . . 18
2.7 Kết luận chương . . . 19

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG . . . 20
3.1 Kiến trúc tổng thể . . . 20
3.2 Thiết kế phần cứng . . . 20
3.2.1 Node Root . . . 22
3.2.2 Node Leaf và khối nguồn . . . 22
3.2.3 Tổng hợp linh kiện . . . 22
3.3 Thiết kế firmware trên mạng mesh . . . 24
3.3.1 Cơ chế phân mảnh và cấu trúc gói chunk . . . 24
3.3.2 Firmware node Leaf . . . 25
3.3.3 Firmware node Root . . . 27
3.3.4 Các tham số cấu hình chính . . . 29
3.4 Thiết kế giao tiếp và phần mềm trên máy tính . . . 30
3.4.1 Giao thức truyền dữ liệu qua UART . . . 30
3.4.2 Middleware trên máy tính . . . 31
3.4.3 Web Dashboard . . . 32
3.5 Kết luận chương . . . 33

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ . . . 34
4.1 Môi trường và phương pháp thử nghiệm . . . 34
4.2 Quá trình đưa phần cứng vào hoạt động . . . 34
4.3 Kết quả với một node Leaf . . . 34
4.4 Kết quả với nhiều node Leaf . . . 35
4.4.1 Hai Leaf và lỗi một vùng đệm ghép . . . 35

4.4.2 Ba Leaf chạy pin và hiện tượng đứng mạng 36
4.4.3 Kết quả ở chu kỳ chụp 600 ms 36
4.5 Đo thời gian tự phục hồi 37
4.6 Các cải tiến ổn định và kết quả ban đầu 38
4.7 Kết luận chương 39

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 40
5.1 Kết luận 40
5.1.1 Kết quả đạt được 40
5.1.2 Hạn chế 41
5.2 Hướng phát triển 41
5.2.1 Hướng phát triển ngắn hạn 41
5.2.2 Hướng phát triển dài hạn 42

TÀI LIỆU THAM KHẢO 44

PHỤ LỤC A. MÃ NGUỒN VÀ TÀI NGUYÊN DỰ ÁN A-1

DANH MỤC HÌNH VẼ

Hình 2.1. Phân bổ tác vụ trên hai lõi ESP32 trong lõi Arduino-ESP32 16

Hình 3.1. Sơ đồ khối tổng thể của hệ thống 21
Hình 3.2. Sơ đồ khối phần cứng của một Leaf 23
Hình 3.3. Cấu trúc gói chunk trên mạng mesh và khung UART 25
Hình 3.4. Lưu đồ chụp và gửi ảnh tại Leaf 26
Hình 3.5. Lưu đồ nhận và ghép ảnh tại Root 28
Hình 3.6. Lưu đồ tách khung tại middleware 31
Hình 3.7. Giao diện Web Dashboard: (a) tab Live; (b) tab So sánh 32

Hình 4.1. So sánh FPS tổng theo số Leaf ở hai chu kỳ chụp (mức 300 ms là giá trị khi mạng còn chạy) 37

DANH MỤC BẢNG BIỂU

Bảng 1.1. Diễn giải các nhóm nhiệm vụ của đề tài 3

Bảng 2.1. So sánh MANET, VANET và FANET 10

Bảng 3.1. Vai trò và phần cứng các node 21
Bảng 3.2. Danh sách linh kiện chính 23
Bảng 3.3. Cấu trúc header chunk 16 byte 24
Bảng 3.4. Các tham số cấu hình chính của firmware 29
Bảng 3.5. Cấu trúc khung UART 30

Bảng 4.1. Các sự cố khi bring-up và cách xử lý 35
Bảng 4.2. Kết quả thử nghiệm ở chu kỳ chụp 600 ms 36
Bảng 4.3. Diễn biến phiên đo tối 29/09 (giờ UTC) 37

DANH MỤC CÁC CHỮ VIẾT TẮT

AGPS	Assisted Global Positioning System
AODV	Ad hoc On-Demand Distance Vector
AP	Access Point
API	Application Programming Interface
B.A.T.M.A.N	Better Approach To Mobile Ad-hoc Networking
COM	Communication port
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
CSV	Comma-Separated Values
DC	Direct Current
DGPS	Differential Global Positioning System
DSDV	Destination-Sequenced Distance Vector
DSR	Dynamic Source Routing
ELP	Echo Location Protocol
ESP-IDF	Espressif IoT Development Framework
FANET	Flying Ad-hoc Network
FPS	Frames Per Second
GPS	Global Positioning System
GPSR	Greedy Perimeter Stateless Routing
ID	Identifier
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
IoT	Internet of Things
IP	Internet Protocol
IWDT	Interrupt Watchdog Timer
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
LoS	Line of Sight

LRU	Least Recently Used
lwIP	Lightweight IP
MANET	Mobile Ad-hoc Network
NACK	Negative Acknowledgement
OGM	Originator Message
OLSR	Optimized Link State Routing
OTA	Over-The-Air
PCB	Printed Circuit Board
PSRAM	Pseudo-Static Random Access Memory
RFC	Request for Comments
RSSI	Received Signal Strength Indicator
RTOS	Real-Time Operating System
SIM	Subscriber Identity Module
SMP	Symmetric Multiprocessing
SoC	System on Chip
SPI	Serial Peripheral Interface
SPIFFS	SPI Flash File System
SRAM	Static Random Access Memory
STA	Station
STT	Số thứ tự
SVG	Scalable Vector Graphics
TBRPF	Topology Broadcast based on Reverse-Path Forwarding
TCP	Transmission Control Protocol
TQ	Transmit Quality
TWDT	Task Watchdog Timer
TX	Transmit
UART	Universal Asynchronous Receiver-Transmitter
UAV	Unmanned Aerial Vehicle
USB	Universal Serial Bus
UTC	Coordinated Universal Time
UWB	Ultra-Wideband
VANET	Vehicular Ad-hoc Network
WMN	Wireless Mesh Network

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày bối cảnh và lý do lựa chọn đề tài, mục tiêu, nhiệm vụ, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện và bố cục của báo cáo. Các yêu cầu của đề tài được diễn giải thành chức năng, thông số kỹ thuật và giới hạn cụ thể để làm căn cứ đánh giá ở các chương sau. Nội dung của chương là cơ sở để các chương tiếp theo đi vào lý thuyết, thiết kế và kết quả thực nghiệm của hệ thống.

1.1 Đặt vấn đề

Trong nhiều hệ thống drone và robot hiện nay, các thiết bị kết nối với trạm điều khiển theo mô hình WiFi hình sao (star topology), tức là mọi thiết bị đều phải liên kết trực tiếp với một bộ định tuyến hoặc điểm truy cập trung tâm. Khi thiết bị di chuyển ra xa vùng phủ của bộ định tuyến, hoặc khi bộ định tuyến gặp sự cố, liên kết của thiết bị với trạm bị gián đoạn. Giải pháp thay thế bằng mạng di động 4G đòi hỏi thẻ SIM (Subscriber Identity Module) và hạ tầng nhà mạng, vốn thường không có ở những nơi drone và robot cần làm việc như vùng thiên tai, hầm mỏ hay khu vực rừng núi.

Mạng mesh không dây khắc phục hạn chế này bằng cách cho mỗi node vừa gửi dữ liệu của chính nó vừa chuyển tiếp dữ liệu cho các node lân cận. Nhờ đó, drone hoặc robot nằm ngoài tầm phát trực tiếp của trạm vẫn gửi được dữ liệu qua một chuỗi liên kết đa chặng (multi-hop) (Akyildiz và cộng sự, 2005; Bekmezci và cộng sự, 2013). Các vi điều khiển tự tạo thành một mạng vô tuyến cục bộ riêng, không cần SIM và không cần Internet. Giả sử một robot hoạt động trong tòa nhà và tường chắn tín hiệu trực tiếp tới trạm; robot vẫn gửi được ảnh nếu có một node trung gian đặt ở hành lang nằm trong tầm phát của cả robot và trạm.

Bài toán đặt ra hai thách thức kỹ thuật chính. Thách thức thứ nhất là định tuyến dữ liệu qua nhiều chặng trong khi topology của mạng thay đổi theo vị trí của các node. Thách thức thứ hai là truyền dữ liệu ảnh có kích thước lớn qua các liên kết vô tuyến có băng thông hạn chế, trong khi mỗi bản tin của thư viện mesh trên ESP32 chỉ mang được khoảng 1400 byte (painlessMesh, n.d.).

Đề tài giải quyết bài toán bằng một mạng WiFi Mesh dùng vi điều khiển ESP32 giá thấp và thư viện painlessMesh. Các node lá (Leaf) mang camera đóng vai trò drone

hoặc robot thu thập hình ảnh, còn node gốc (Root) nối với máy tính đóng vai trò trạm giám sát. Ảnh định dạng JPEG (Joint Photographic Experts Group) được chia thành nhiều phần nhỏ, truyền qua mạng mesh, ghép lại tại Root và hiển thị trên một giao diện web cùng với các thông số chất lượng mạng.

Về ý nghĩa thực tiễn, đề tài tạo ra một mô hình thử nghiệm giá thấp để đánh giá khả năng truyền ảnh qua mạng mesh trước khi đưa hệ thống lên drone hoặc robot thật. Mô hình phục vụ người phát triển các hệ thống này: thay vì chỉ dựa vào thông số của nhà sản xuất, người phát triển có thể đo trực tiếp thông lượng, số khung trên giây, tỉ lệ hủy khung, độ trễ và thời gian tự phục hồi trên phần cứng thật. Các phép đo ở Chương 4 cho thấy cách mô hình này giúp xác định nút thắt của hệ thống, cụ thể là số vùng đệm ghép ảnh và tải ghi dữ liệu qua giao tiếp nối tiếp UART (Universal Asynchronous Receiver-Transmitter) tại Root.

1.2 Mục tiêu của đề tài

Mục tiêu của đề tài được xác định ở hai mức. Mục tiêu tổng quát định hướng cho toàn bộ quá trình thực hiện, còn các mục tiêu cụ thể được dùng làm tiêu chí đánh giá kết quả. Các tiêu chí này được định nghĩa thành công thức ở mục 2.6 và được đo ở Chương 4.

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là thiết kế, thi công và đánh giá một mạng WiFi Mesh nhiều node có khả năng truyền ảnh từ các node di động về trạm giám sát theo thời gian thực. Bên cạnh chức năng truyền ảnh, hệ thống phải cung cấp công cụ đo lường để đánh giá chất lượng mạng ngay trong quá trình hoạt động. Các node trong đề tài mô phỏng drone và robot, nên mục tiêu tập trung vào khối truyền thông chứ không vào khối bay hay cơ khí.

1.2.2 Mục tiêu cụ thể

Vì đây là đồ án thuộc chuyên ngành Điện tử – Viễn thông, trọng tâm đánh giá là chất lượng mạng vô tuyến chứ không phải khả năng bay hay cơ khí của drone. Đề tài đặt ra ba tiêu chí đánh giá chính. Tiêu chí thứ nhất là thông lượng (throughput), gồm giá trị thông lượng và mức suy giảm của nó khi dữ liệu đi qua nhiều hop. Tiêu chí thứ hai là độ trễ (latency), tức khoảng thời gian từ lúc chụp ảnh đến lúc ảnh được

ghép xong tại trạm giám sát. Tiêu chí thứ ba là thời gian tự phục hồi (self-healing), tức khoảng thời gian mạng khôi phục đường truyền khi một node trung gian mất nguồn hoặc ra khỏi vùng phủ sóng.

1.3 Nhiệm vụ của đề tài

Để đạt được các mục tiêu trên, yêu cầu của đề tài được diễn giải thành sáu nhóm nhiệm vụ, bao phủ từ nghiên cứu nền tảng phần mềm, phần cứng, cơ chế truyền ảnh, phần mềm trên máy tính đến kịch bản thử nghiệm. Với mỗi nhóm, Bảng 1.1 nêu chức năng cần đạt, thông số kỹ thuật được chọn và giới hạn trong phạm vi đồ án. Các nhóm nhiệm vụ này cũng quyết định cấu trúc của Chương 3 và Chương 4.

Bảng 1.1. Diễn giải các nhóm nhiệm vụ của đề tài

STT	Nhóm nhiệm vụ	Chức năng, thông số và giới hạn
1	Nền tảng phần mềm	Tìm hiểu cơ chế tác vụ của FreeRTOS và cách painlessMesh tự tổ chức mạng cây; thiết kế mô hình một Root nhận dữ liệu từ nhiều Leaf.
2	Phần cứng và nguồn	Một Root ESP32 DevKitC cấp nguồn qua cổng USB (Universal Serial Bus); ba Leaf ESP32-CAM OV3660, mỗi Leaf dùng pin 18650 và module tăng áp 5 V; đo dòng tiêu thụ và đánh giá độ ổn định khi truyền.
3	Truyền ảnh qua mesh	Chia ảnh JPEG thành chunk tối đa 1024 byte kèm header 16 byte; Root ghép tối đa bốn khung song song và hủy khung thiếu chunk sau 2 giây; chưa có cơ chế gửi lại.
4	Thu nhận dữ liệu trên máy tính	Root gửi khung nhị phân có mã kiểm tra CRC16 (Cyclic Redundancy Check) qua UART 921600 bps; middleware Node.js phát dữ liệu tới trình duyệt bằng Socket.IO trên nền WebSocket.
5	Giao diện giám sát	Web Dashboard hiển thị ảnh từng Leaf, topology, cường độ tín hiệu RSSI (Received Signal Strength Indicator), độ trễ, thông lượng, số khung trên giây, tỉ lệ hủy khung và xuất dữ liệu ra tệp CSV (Comma-Separated Values).
6	Kịch bản thử nghiệm	Thay đổi số Leaf từ một đến ba, số hop và tải dữ liệu qua chu kỳ chụp 300 ms hoặc 600 ms; đánh giá thông lượng, độ trễ, tỉ lệ hủy khung, khả năng tự phục hồi và khả năng mở rộng.

1.4 Đối tượng và phạm vi nghiên cứu

Mục này xác định đối tượng mà đề tài tập trung nghiên cứu và giới hạn phạm vi thực hiện. Đối tượng và phạm vi này là cơ sở để xây dựng các kịch bản thử nghiệm được trình bày ở Chương 4. Những nội dung nằm ngoài phạm vi được nêu rõ để tránh mở rộng nhận xét vượt quá điều kiện đã thử nghiệm.

1.4.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu là mạng mesh WiFi tạo bởi các vi điều khiển ESP32 sử dụng thư viện painlessMesh, cùng cơ chế truyền ảnh JPEG qua mạng này. Phần cứng thử nghiệm gồm một Root dùng board ESP32-WROOM-32D DevKitC và ba Leaf dùng board ESP32-CAM với camera OV3660. Các node sử dụng anten mạch in PCB (Printed Circuit Board) tích hợp sẵn trên board, không sử dụng anten rời.

1.4.2 Phạm vi nghiên cứu

Đề tài không chế tạo khung drone hay robot. Chuyển động của drone được mô phỏng bằng cách cầm tay hoặc đặt các node ở những vị trí cố định khác nhau trên mặt đất, không sử dụng xe điều khiển từ xa. Ba Leaf có cấu hình phần cứng giống nhau; vai trò chuyển tiếp không gắn với một phần cứng riêng mà do mạng tự hình thành tùy theo vị trí và cường độ tín hiệu tại thời điểm thử nghiệm. Với một Root và ba Leaf, mạng có thể tạo chuỗi tối đa ba hop để đo thông lượng và độ trễ theo số hop.

1.5 Phương pháp thực hiện

Đề tài được thực hiện theo phương pháp lặp. Sau khi nghiên cứu cơ sở lý thuyết, đề tài xây dựng firmware và phần mềm, rồi đưa từng board vào hoạt động (bring-up). Tiếp theo, hệ thống được thử nghiệm với số node tăng dần và số liệu được thu thập bằng Web Dashboard; mỗi khi xảy ra lỗi, nguyên nhân được chẩn đoán, khắc phục và hệ thống được đo lại.

Về công cụ, firmware được phát triển trên môi trường lập trình Arduino IDE (Integrated Development Environment) 2.3.10 với gói board esp32 của Espressif phiên bản 2.0.17, được hạ từ phiên bản 3.3.11 theo khuyến nghị của painlessMesh, và thư viện Alteriom/painlessMesh 2.0.3. Phần mềm trên máy tính sử dụng Node.js, giao diện giám sát sử dụng React và Tailwind CSS. Mã nguồn được quản lý bằng Git trên GitHub.

Trong quá trình thực hiện, đề tài từng thử nghiệm song song một phiên bản dùng giao diện lập trình API (Application Programming Interface) ESP-WIFI-MESH chính thức của Espressif trên nền ESP-IDF (Espressif IoT Development Framework). Ngày 29/09/2026, đề tài chốt sử dụng painlessMesh làm phiên bản chính thức vì danh sách nhiệm vụ đã xác định rõ giao thức này. Phiên bản ESP-IDF được giữ lại và đưa vào hướng phát triển ở Chương 5.

1.6 Bố cục báo cáo

Báo cáo được tổ chức thành năm chương, đi từ yêu cầu của đề tài tới lý thuyết, thiết kế, kết quả đo và kết luận. Chương 1 giới thiệu bài toán, mục tiêu, nhiệm vụ, đối tượng, phạm vi và phương pháp thực hiện. Chương 2 trình bày cơ sở lý thuyết về mạng mesh không dây, các giao thức định tuyến, mạng ad-hoc cho thiết bị bay, ESP-WIFI-MESH và painlessMesh, FreeRTOS trên ESP32, nền tảng phần cứng và các chỉ số đánh giá hiệu năng mạng.

Dựa trên các cơ sở đó, Chương 3 trình bày thiết kế hệ thống theo thứ tự từ sơ đồ khối tổng thể tới phần cứng các node, firmware trên mạng mesh và phần mềm trên máy tính gồm giao thức UART, middleware và Web Dashboard. Chương 4 trình bày quá trình thử nghiệm, kết quả đo và các nhận xét rút ra từ số liệu. Cuối cùng, Chương 5 tổng kết các kết quả đạt được, các hạn chế và hướng phát triển của đề tài.

1.7 Kết luận chương

Chương 1 đã xác định bài toán liên lạc cho drone và robot khi không có hạ tầng mạng cố định và đề xuất hướng giải quyết bằng mạng WiFi Mesh dùng ESP32. Các yêu cầu của đề tài đã được diễn giải thành sáu nhóm nhiệm vụ với thông số và giới hạn cụ thể, cùng phạm vi thử nghiệm trên mặt đất với một Root và ba Leaf. Chương 2 tiếp theo trình bày cơ sở lý thuyết làm nền cho thiết kế và đánh giá hệ thống.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các cơ sở lý thuyết mà thiết kế ở Chương 3 và các phép đo ở Chương 4 dựa vào. Nội dung đi từ khái niệm mạng mesh không dây và các giao thức định tuyến cho mạng ad-hoc, trong đó có mạng ad-hoc cho thiết bị bay, đến hai giải pháp mesh chạy trên ESP32 là ESP-WIFI-MESH và painlessMesh. Tiếp theo, chương trình bày hệ điều hành thời gian thực FreeRTOS trên ESP32, vì cách FreeRTOS phân chia tác vụ quyết định việc firmware có xử lý kịp lưu lượng mạng hay không. Hai phần cuối giới thiệu nền tảng phần cứng và định nghĩa các chỉ số dùng để đánh giá hiệu năng mạng.

2.1 Mạng mesh không dây

Mục này trình bày khái niệm, các đặc điểm cốt lõi của mạng mesh không dây và so sánh mô hình mesh với các mô hình mạng truyền thống. Đây là nền tảng để giải thích vì sao đề tài chọn mô hình mesh cho bài toán kết nối drone và robot. Các khái niệm về truyền đa chặng và tự phục hồi được định nghĩa ở đây cũng được dùng xuyên suốt các chương sau.

2.1.1 Khái niệm và đặc điểm

Theo khảo sát của Akyildiz và cộng sự (2005), mạng mesh không dây (Wireless Mesh Network – WMN) gồm hai thành phần: mesh router, thường ít di chuyển và tạo thành xương sống của mạng, và mesh client, có thể di động. Các mesh router cung cấp khả năng định tuyến đa chặng (multi-hop), tức là dữ liệu được chuyển tiếp lần lượt qua nhiều node trung gian; mỗi lần dữ liệu đi qua một liên kết giữa hai node được tính là một hop. Nhờ cơ chế này, mạng tự mở rộng vùng phủ mà không cần mỗi node nối trực tiếp tới hạ tầng trung tâm, nên drone hay robot không cần nằm trong tầm phát của trạm gốc mà chỉ cần có một chuỗi liên kết giữa các node lân cận.

Đề tài khai thác ba đặc điểm cốt lõi của WMN. Thứ nhất, mạng có khả năng tự tổ chức và tự cấu hình, nghĩa là các node tự phát hiện nhau và tự hình thành liên kết mà không cần cấu hình thủ công. Thứ hai, mạng hỗ trợ truyền đa chặng thông qua các node trung gian. Thứ ba, mạng có khả năng tự phục hồi (self-healing), tức là tự

tìm đường thay thế khi một liên kết bị mất. Ngoài ra, WMN còn có khả năng kết nối với các loại mạng khác, nhưng khả năng này nằm ngoài phạm vi đề tài.

2.1.2 So sánh với các mô hình mạng khác

Trong mô hình bus, các thiết bị dùng chung một trục truyền dẫn vật lý, khi trục hỏng thì toàn mạng ngừng hoạt động. Mô hình này không áp dụng cho mạng mesh không dây vì không có trục chung, mỗi liên kết là độc lập giữa từng cặp node. Mô hình hình sao phụ thuộc hoàn toàn vào node trung tâm, và đây chính là hạn chế mà đề tài hướng tới giải quyết. Trong mô hình mesh, mỗi node có thể có nhiều liên kết với các node lân cận; khi mật độ liên kết đủ cao, mạng vẫn giữ được kết nối toàn cục dù một node đơn lẻ bị lỗi, và đây là nền tảng của khái niệm tự phục hồi.

Khi các drone xếp nối tiếp nhau thành một chuỗi, mạng thực chất có dạng đường thẳng (daisy-chain), một dạng suy biến của mạng mesh. Nếu drone ở giữa chuỗi mất nguồn, các drone phía ngoài không bị hỏng về mặt vật lý nhưng sẽ bị cô lập khỏi trạm nếu drone đó là cầu nối duy nhất. Ngược lại, khi mật độ node đủ dày, các drone phía ngoài có thể tìm đường vòng qua node khác. Mô hình cây (tree) là một dạng suy biến khác của mesh và là mô hình mà ESP-WIFI-MESH và painlessMesh sử dụng, như trình bày ở mục 2.3.

2.2 Các giao thức định tuyến cho mạng ad-hoc

Định tuyến là chức năng cốt lõi của mạng mesh vì nó quyết định dữ liệu đi qua những node nào để tới đích. Mục này trình bày cách phân loại các giao thức định tuyến cho mạng ad-hoc di động, giao thức B.A.T.M.A.N và mạng ad-hoc dành cho thiết bị bay. Nội dung này giúp xác định giao thức nào chạy được trên vi điều khiển ESP32 và đề tài đang đứng ở đâu so với các hướng nghiên cứu hiện có.

2.2.1 Phân loại giao thức định tuyến MANET

Các giao thức định tuyến cho mạng ad-hoc di động (Mobile Ad-hoc Network – MANET) thường được chia thành hai nhóm chính là nhóm phản ứng và nhóm chủ động. Nhóm phản ứng (reactive hay on-demand) chỉ tìm đường khi có dữ liệu cần gửi; tiêu biểu là AODV (Ad hoc On-Demand Distance Vector), được chuẩn hóa thành tài liệu RFC (Request for Comments) 3561 (Perkins và cộng sự, 2003). Vì chỉ khởi

tạo quá trình dò đường khi cần, AODV tiết kiệm băng thông dành cho bản tin điều khiển, đổi lại gói tin đầu tiên tới một đích mới phải chờ quá trình dò đường hoàn tất.

Nhóm chủ động (proactive) liên tục cập nhật bảng định tuyến cho toàn mạng nên đường đi luôn có sẵn khi cần gửi. Tiêu biểu cho nhóm này là OLSR (Optimized Link State Routing), được chuẩn hóa thành RFC 3626 (Clausen và Jacquet, 2003), sử dụng cơ chế Multipoint Relay để giảm số node phải phát tràn bản tin điều khiển. Một giao thức chủ động khác là DSDV (Destination-Sequenced Distance Vector) (Perkins và Bhagwat, 1994), một trong những giao thức MANET sớm nhất, sử dụng số thứ tự để tránh vòng lặp định tuyến.

Hai nhóm trên đánh đổi giữa chi phí bản tin điều khiển và độ trễ khi bắt đầu gửi dữ liệu. Tuy nhiên, cả AODV, OLSR và DSDV đều được thiết kế cho tầng IP (Internet Protocol) trên các hệ điều hành như Linux và không có bản cài đặt sẵn cho vi điều khiển ESP32. Vì vậy, đề tài không sử dụng trực tiếp các giao thức này mà dùng chúng làm cơ sở để so sánh với cơ chế định tuyến của painlessMesh.

2.2.2 Giao thức B.A.T.M.A.N

B.A.T.M.A.N (Better Approach To Mobile Ad-hoc Networking) hoạt động ở tầng 2 và biến mạng mesh thành một switch ảo (Open Mesh, n.d.). Điểm khác biệt chính so với OLSR và AODV là mỗi node không cần biết toàn bộ topology, chỉ cần biết node lân cận nào là node kế tiếp tốt nhất trên đường tới từng đích. Mỗi node phát định kỳ bản tin OGM (Originator Message); OGM đi qua đường tốt sẽ đến nhanh hơn và ít bị mất hơn, nên đường đó được chọn. Phiên bản IV dùng chỉ số TQ (Transmit Quality) để xử lý liên kết bất đối xứng và thêm trường previous sender để loại bỏ OGM quay vòng, còn phiên bản V tách chức năng đo chất lượng liên kết cục bộ sang bản tin ELP (Echo Location Protocol) để giảm tải xử lý.

Do việc chọn đường dựa trên các OGM phát định kỳ, B.A.T.M.A.N chỉ đổi đường sau khi đã nhận đủ OGM qua đường mới, nên tốc độ phản ứng khi topology thay đổi phụ thuộc vào chu kỳ phát OGM. Giao thức được cài đặt dưới dạng module batman-adv trong nhân Linux và cấu hình bằng công cụ batctl (Open Mesh, n.d.). Vì vậy, B.A.T.M.A.N phù hợp với máy tính nhúng chạy Linux hơn là vi điều khiển ESP32.

2.2.3 Mạng ad-hoc cho thiết bị bay

Mạng ad-hoc cho thiết bị bay (Flying Ad-hoc Network – FANET) là mạng ad-hoc mà các node là thiết bị bay không người lái (Unmanned Aerial Vehicle – UAV). Theo Bekmezci và cộng sự (2013), FANET được xem là một dạng đặc biệt của MANET và của mạng ad-hoc cho phương tiện giao thông (Vehicular Ad-hoc Network – VANET). Một hệ thống chỉ được gọi là FANET khi thỏa hai điều kiện: hệ thống phải có nhiều UAV, vì một UAV đơn lẻ không tạo thành mạng, và các UAV phải liên lạc với nhau qua mạng ad-hoc giữa chúng. Nếu mọi UAV chỉ trao đổi dữ liệu qua trạm mặt đất hoặc vệ tinh thì hệ thống đó không phải là FANET (Bekmezci và cộng sự, 2013).

FANET ra đời để khắc phục nhược điểm của kiến trúc lấy hạ tầng làm trung tâm, trong đó mỗi UAV liên lạc trực tiếp với trạm mặt đất hoặc vệ tinh. Kiến trúc này đòi hỏi mỗi UAV mang phần cứng liên lạc đắt và nặng, đồng thời dễ mất kết nối khi UAV bay ra ngoài vùng phủ hoặc bị địa hình che khuất. Trong FANET, chỉ một số UAV cần nối với hạ tầng, các UAV còn lại chuyển tiếp dữ liệu qua nhau theo cùng nguyên lý đa chặng của WMN đã trình bày ở mục 2.1 (Akyildiz và cộng sự, 2005).

Bekmezci và cộng sự (2013) nêu nhiều kịch bản ứng dụng của FANET, trong đó ba kịch bản liên quan nhất tới đề tài là mở rộng vùng hoạt động, tăng độ tin cậy và phối hợp bầy UAV. Ở kịch bản mở rộng vùng hoạt động, các UAV tạo thành chuỗi liên kết đa chặng để vươn ra ngoài vùng phủ của trạm mặt đất, kể cả khi đường truyền thẳng bị nhà cao tầng hay núi che chắn. Ở kịch bản tăng độ tin cậy, một UAV mất kết nối với hạ tầng vẫn duy trì được liên lạc thông qua các UAV khác, còn ở kịch bản bầy UAV, nhiều UAV nhỏ phối hợp với nhau để thực hiện một nhiệm vụ phức tạp. Kịch bản mở rộng vùng hoạt động gần nhất với đề tài, vì trong đề tài một Leaf ở xa có thể gửi ảnh về Root thông qua một node trung gian.

Bảng 2.1 tổng hợp các điểm khác biệt giữa MANET, VANET và FANET theo các tiêu chí về di động, topology, truyền sóng, năng lượng và định vị. Bảng được kẻ lại từ Bảng 1 trong khảo sát của Bekmezci và cộng sự (2013), trong đó LoS (Line of Sight) chỉ đường truyền thẳng không bị che chắn, còn AGPS (Assisted GPS), DGPS (Differential GPS) và IMU (Inertial Measurement Unit) là các phương pháp định vị bổ trợ cho GPS (Global Positioning System). Các khác biệt này giải thích vì sao giao thức thiết kế cho MANET không phải lúc nào cũng phù hợp với FANET.

Bảng 2.1. So sánh MANET, VANET và FANET

Nguồn: tổng hợp theo Bảng 1 của Bekmezci và cộng sự (2013)

Tiêu chí	MANET	VANET	FANET
Mức độ di động	Thấp	Cao	Rất cao
Mô hình di chuyển	Ngẫu nhiên	Theo đường giao thông	Theo kế hoạch bay hoặc mô hình riêng cho UAV tự hành
Mật độ node	Thấp	Cao	Rất thấp
Thay đổi topology	Chậm	Nhanh	Nhanh
Truyền sóng	Gần mặt đất, thường không có LoS	Gần mặt đất, thường không có LoS	Trên cao, thường có LoS
Năng lượng	Cần giao thức tiết kiệm năng lượng	Không phải vấn đề chính	Quan trọng với mini-UAV
Năng lực tính toán	Hạn chế	Cao	Cao
Định vị	GPS	GPS, AGPS, DGPS	GPS, AGPS, DGPS, IMU

Theo Bekmezci và cộng sự (2013), tốc độ của UAV có thể từ 30 đến 460 km/h, nên topology của FANET thay đổi liên tục. Ngoài chuyển động, topology còn thay đổi khi một UAV hỏng, khi bổ sung UAV mới hoặc khi chất lượng liên kết dao động. Khoảng cách giữa các node FANET cũng lớn hơn nhiều so với MANET và VANET, do đó tầm phát của mỗi node cần dài hơn. Về lưu lượng, FANET phải hỗ trợ đồng thời hai kiểu: lưu lượng ngang hàng giữa các UAV để phối hợp, và lưu lượng hội tụ (convergecast), trong đó nhiều UAV cùng gửi dữ liệu cảm biến về một trạm. Đề tài thuộc trường hợp hội tụ, vì nhiều Leaf cùng gửi ảnh về một Root.

Bekmezci và cộng sự (2013) nêu năm yêu cầu thiết kế chính của FANET là khả năng thích nghi, khả năng mở rộng, độ trễ, ràng buộc của nền tảng UAV và băng thông. Khả năng thích nghi đòi hỏi mạng tự điều chỉnh khi node di chuyển, hỏng hoặc được thêm mới; trong đề tài, yêu cầu này tương ứng với cơ chế tự phục hồi của painlessMesh và phép đo thời gian tự phục hồi ở Chương 4. Khả năng mở rộng đòi hỏi hiệu năng không suy giảm nhiều khi số UAV tăng, và được đề tài đánh giá bằng các phép đo với một, hai và ba Leaf. Yêu cầu về độ trễ phụ thuộc ứng dụng, nên đề tài đưa độ trễ từ lúc chụp đến lúc ghép xong ảnh vào nhóm chỉ số giám sát. Ràng buộc

về khối lượng và kích thước phần cứng là lý do đề tài dùng module ESP32-CAM nhỏ, nhẹ. Cuối cùng, yêu cầu truyền ảnh qua kênh vô tuyến dễ lỗi là lý do ảnh phải được phân mảnh và tốc độ khung phải được giới hạn theo khả năng xử lý của Root.

Về định tuyến, các thử nghiệm FANET đầu tiên dùng lại giao thức MANET thuộc nhóm chủ động như OLSR và TBRPF (Topology Broadcast based on Reverse-Path Forwarding) hoặc nhóm phản ứng như DSR (Dynamic Source Routing) và AOD-V (Bekmezci và cộng sự, 2013). Khi topology thay đổi nhanh, việc duy trì bảng định tuyến theo kiểu chủ động trở nên kém hiệu quả, còn việc tìm lại đường trước khi gửi theo kiểu phản ứng lại tốn thời gian. Bekmezci và cộng sự (2013) dẫn một nghiên cứu mô phỏng, trong đó GPSR (Greedy Perimeter Stateless Routing), một giao thức định tuyến dựa trên vị trí, cho kết quả tốt hơn các giao thức chủ động và phản ứng trong FANET, dù độ tin cậy của nhóm này giảm khi mạng thưa. Nhóm định tuyến phân cấp, chia mạng thành các cụm có node chủ cụm, phù hợp khi vùng nhiệm vụ rộng và số UAV lớn.

Đối chiếu với các nội dung trên, painlessMesh tự tổ chức thành cấu trúc cây và không sử dụng thông tin vị trí của node. Các node trong đề tài được đặt cố định hoặc cầm tay trên mặt đất, không bay, nên đề tài không khẳng định hệ thống là một FANET. Thay vào đó, đề tài xây dựng và thử nghiệm một mô hình truyền thông đa chặng theo hướng FANET cho drone và robot, và xem định tuyến dựa trên vị trí là một hướng có thể thử nghiệm khi bổ sung module định vị.

2.3 ESP-WIFI-MESH và thư viện painlessMesh

Trên nền tảng ESP32, có hai lựa chọn chính để xây dựng mạng mesh WiFi là giao thức ESP-WIFI-MESH của Espressif và thư viện mã nguồn mở painlessMesh. Mục này trình bày nguyên lý hoạt động và các giới hạn của từng lựa chọn. Các giới hạn của painlessMesh được trình bày chi tiết hơn vì đây là thư viện đề tài sử dụng.

2.3.1 ESP-WIFI-MESH

ESP-WIFI-MESH là giao thức mesh chính thức của Espressif (Espressif Systems, n.d.-b). Giao thức tận dụng khả năng hoạt động đồng thời ở chế độ station (STA) và access point (AP) của chip: một node vừa kết nối lên node cha như một station, vừa nhận kết nối từ các node con như một access point. Cơ chế này tạo ra topology dạng cây nhiều lớp, tương ứng với các vai trò Root, node chuyển tiếp và Leaf trong

đề tài. Theo tài liệu của Espressif, mạng có thể mở rộng tới quy mô lý thuyết khoảng 1000 node (Espressif Systems, n.d.-c) và hoạt động đa chặng mà không cần bộ định tuyến.

Tài liệu ESP-IDF mô tả hai kịch bản tự phục hồi (Espressif Systems, n.d.-b). Khi node Root hỏng, các node ở lớp thứ hai tiến hành bầu chọn Root mới dựa trên RSSI tới bộ định tuyến. Khi một node cha trung gian hỏng, các node con thử kết nối lại, sau đó tự chọn node cha mới trong tầm phủ sóng; quá trình này có thể làm mạng tăng thêm một lớp, kéo theo số hop và độ trễ tăng. ESP-WIFI-MESH còn có cơ chế điều khiển luồng lên (receiving window) và hàng đợi có giới hạn để chống tràn bộ đệm.

2.3.2 Thư viện painlessMesh

painlessMesh là thư viện mã nguồn mở cho nền tảng Arduino, cho phép các ESP8266 và ESP32 tự tổ chức thành mạng mesh mà không cần bộ định tuyến trung tâm (painlessMesh, n.d.). Mạng có dạng cây, không có vòng lặp; các node trao đổi bản tin dạng JSON (JavaScript Object Notation) qua kết nối TCP (Transmission Control Protocol) giữa từng cặp node cha – con. Ứng dụng gửi dữ liệu bằng hàm sendSingle tới một node cụ thể hoặc sendBroadcast tới toàn mạng. Thư viện duy trì một đồng hồ chung toàn mạng, truy xuất bằng hàm getNodeTime với đơn vị micro giây, có độ chính xác đồng bộ cỡ vài mili giây và cuộn vòng sau khoảng 71 phút.

Thư viện có bốn giới hạn quan trọng đối với bài toán truyền ảnh (painlessMesh, n.d.). Thứ nhất, mỗi bản tin bị giới hạn kích thước khoảng 1400 byte, nên một ảnh JPEG phải được chia thành nhiều phần. Thứ hai, bản tin broadcast được chuyển tiếp lần lượt qua từng cạnh TCP có xác nhận, nên một ảnh gửi broadcast bị nhân lên nhiều lần trên cùng một miền đụng độ. Thứ ba, khi một node mất nguồn đột ngột, kết nối TCP không được đóng theo trình tự thông thường, nên việc phát hiện mất node phụ thuộc vào thời gian chờ NODE_TIMEOUT, khoảng 10 giây. Thứ tư, bản painlessMesh gốc có lỗi khiến node ngừng quét tìm mạng sau khi ngắt kết nối WiFi; đề tài sử dụng bản fork Alteriom/painlessMesh (Alteriom, n.d.) đã khắc phục nhóm lỗi này.

Các node trong mạng dùng chung một kênh vô tuyến và truy cập kênh theo cơ chế CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance). Theo mô hình phân tích của Bianchi (2000), trong mạng 802.11 bão hòa, xác suất đụng độ

phụ thuộc vào cửa sổ tranh chấp và số trạm chứ không phụ thuộc độ dài gói. Vì vậy, giảm kích thước mỗi phần dữ liệu không làm giảm đụng độ mà còn làm tăng số khung truyền và phần mào đầu; đây là cơ sở để đề tài chọn kích thước phần dữ liệu gần với giới hạn bản tin của painlessMesh, như trình bày ở Chương 3.

2.4 Hệ điều hành thời gian thực FreeRTOS trên ESP32

Firmware của đề tài chạy trên FreeRTOS, nên cách FreeRTOS chia sẻ bộ xử lý trung tâm (Central Processing Unit – CPU) giữa các tác vụ ảnh hưởng trực tiếp tới việc Root có xử lý kịp dữ liệu mesh hay không. Mục này đi từ các khái niệm chung của FreeRTOS, sang cách FreeRTOS được sửa đổi để chạy trên hai lõi của ESP32 và cơ chế watchdog, rồi gắn các cơ chế đó với hiện tượng quá tải đo được ở Chương 4. Nguồn chính của mục là tài liệu ESP-IDF Programming Guide của Espressif (Espressif Systems, n.d.-e,n) và mã nguồn lõi Arduino-ESP32 (Espressif Systems, 2024).

2.4.1 Tác vụ và các cơ chế giao tiếp

Hệ điều hành thời gian thực (Real-Time Operating System – RTOS) là hệ điều hành dành cho vi điều khiển, bảo đảm các công việc được thực hiện trong giới hạn thời gian xác định. FreeRTOS là một nhân RTOS nhỏ gọn, chạy được trên nhiều dòng vi điều khiển và hệ thống trên chip (System on Chip – SoC) đơn lõi. Trong FreeRTOS, chương trình được chia thành các tác vụ (task); mỗi tác vụ thường là một vòng lặp vô hạn và không bao giờ kết thúc bằng lệnh return.

Tại mỗi thời điểm, một tác vụ ở một trong bốn trạng thái: đang chạy (Running), sẵn sàng (Ready), bị chặn (Blocked) hoặc tạm dừng (Suspended). Tác vụ ở trạng thái Blocked, ví dụ đang chờ dữ liệu từ hàng đợi hoặc chờ một thông báo, không tiêu tốn thời gian CPU. Nhờ đặc điểm này, một tác vụ chỉ chạy khi thực sự có việc, và CPU được dành cho các tác vụ khác trong thời gian chờ.

Các tác vụ trao đổi dữ liệu và đồng bộ với nhau qua hàng đợi (queue), semaphore, mutex, nhóm sự kiện (event group) và thông báo tác vụ (task notification). Hàng đợi dùng để chuyển dữ liệu từ tác vụ này sang tác vụ khác theo thứ tự vào trước ra trước. Thông báo tác vụ là cách nhẹ và nhanh để báo một sự kiện trực tiếp cho một tác vụ, và có thể dùng thay cho semaphore trong nhiều trường hợp.

2.4.2 Bộ lập lịch

FreeRTOS dùng bộ lập lịch ưu tiên cố định, có chiếm quyền (preemptive) và có chia lát thời gian (time slicing) (Espressif Systems, n.d.-e). Bộ lập lịch luôn chọn tác vụ Ready có mức ưu tiên cao nhất để chạy và có thể ngắt tác vụ đang chạy mà không cần tác vụ đó tự nhường CPU. Các tác vụ có cùng mức ưu tiên được luân phiên theo kiểu round-robin tại mỗi ngắt tick.

Ngắt tick là ngắt tuần hoàn có ba nhiệm vụ: tăng bộ đếm tick, chuyển các tác vụ đã hết thời gian chờ sang trạng thái Ready và kích hoạt chuyển ngữ cảnh khi cần chia lát thời gian. Khi không còn tác vụ nào sẵn sàng, tác vụ idle có mức ưu tiên 0 được chạy. Nhiệm vụ của tác vụ idle là giải phóng bộ nhớ của các tác vụ đã bị xóa, nên tác vụ idle cần được chạy định kỳ.

Với tác vụ cần chạy theo chu kỳ cố định, hàm xTaskDelayUntil() phù hợp hơn hàm vTaskDelay(). Lý do là vTaskDelay() tính thời gian chờ tương đối kể từ lúc gọi, nên thời gian xử lý trong mỗi vòng làm chu kỳ bị kéo dài dần. Ngược lại, xTaskDelayUntil() tính thời điểm thức dậy theo mốc tuyệt đối, nên chu kỳ được giữ ổn định.

2.4.3 FreeRTOS trên ESP32 hai lõi

ESP32 có hai lõi xử lý giống nhau và dùng chung bộ nhớ, tức là một hệ đa xử lý đối xứng (Symmetric Multiprocessing – SMP). Espressif sửa đổi FreeRTOS gốc thành phiên bản IDF FreeRTOS để chạy trên cả hai lõi (Espressif Systems, n.d.-e). Trong ESP-IDF, core 0 còn được gọi là PRO_CPU vì thường chạy các tác vụ giao thức như WiFi và Bluetooth, còn core 1 được gọi là APP_CPU vì thường chạy phần ứng dụng.

Một tác vụ có thể được gắn vào một lõi bằng hàm xTaskCreatePinnedToCore() với tham số lõi là 0, 1 hoặc tskNO_AFFINITY nếu cho phép chạy trên cả hai lõi. Cần lưu ý rằng kích thước ngăn xếp của tác vụ trong IDF FreeRTOS được tính bằng byte, không phải bằng word như trong FreeRTOS gốc. Mỗi lõi tự lập lịch độc lập và chọn tác vụ Ready có ưu tiên cao nhất mà nó được phép chạy; hệ quả là hai tác vụ có ưu tiên cao nhất chưa chắc cùng chạy nếu cả hai bị gắn vào cùng một lõi.

Chỉ core 0 thực hiện việc tăng bộ đếm tick của hệ thống, còn mỗi lõi có một tác vụ idle riêng (Espressif Systems, n.d.-e). Trên hệ hai lõi, việc tắt ngắt không còn đủ để bảo vệ vùng găng (critical section), vì lõi còn lại vẫn có thể truy cập cùng dữ

liệu. Do đó, IDF FreeRTOS dùng spinlock kiểu portMUX_TYPE cho vùng găng, và vùng găng phải được giữ ngắn nhất có thể.

2.4.4 Cơ chế watchdog

ESP-IDF cung cấp hai bộ giám sát (watchdog) chính là IWDT và TWDT (Espressif Systems, n.d.-f). IWDT (Interrupt Watchdog Timer) phát hiện trường hợp các trình phục vụ ngắt bị chặn quá lâu trên một lõi. Vì việc chuyển tác vụ của FreeRTOS cũng được thực hiện từ ngắt, khi IWDT hết hạn thì không tác vụ nào khác trên lõi đó được chạy, kể cả tác vụ WiFi và tác vụ idle.

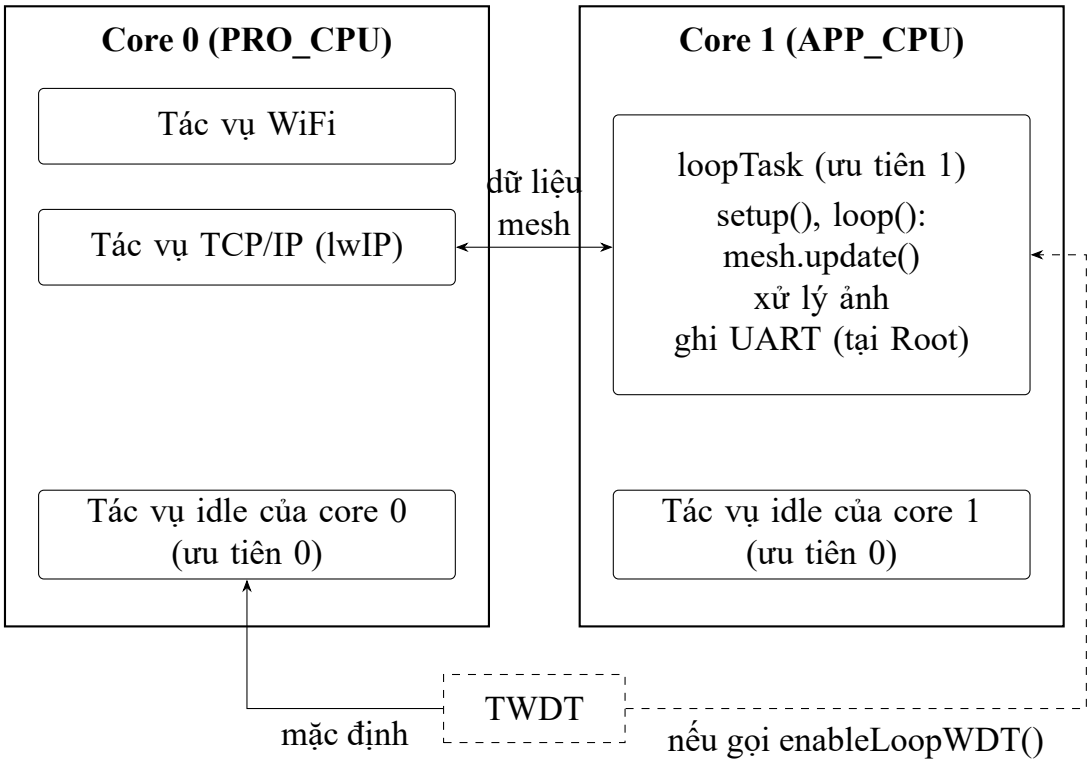
TWDT (Task Watchdog Timer) phát hiện tác vụ chạy quá lâu mà không nhường CPU. Theo tài liệu ESP-IDF, TWDT theo dõi tác vụ idle của các lõi; nếu một tác vụ ưu tiên cao hơn chiếm lõi quá lâu, tác vụ idle của lõi đó không được chạy và TWDT báo lỗi (Espressif Systems, n.d.-f). Các tác vụ khác có thể đăng ký với TWDT bằng hàm esp_task_wdt_add() và phải gọi esp_task_wdt_reset() định kỳ để báo rằng tác vụ vẫn hoạt động bình thường.

Trong lõi Arduino-ESP32, tác vụ chạy hàm loop() không được đăng ký với TWDT theo mặc định: biến loopTaskWDTEnabled được đặt bằng false trong tệp main.cpp, và chương trình phải gọi hàm enableLoopWDT() để bật việc giám sát (Espressif Systems, 2024). Ngoài ra, trong cấu hình sdkconfig biên dịch sẵn của phiên bản 2.0.17, TWDT chỉ theo dõi tác vụ idle của core 0. Vì vậy, một vòng loop() bị chặn lâu trên core 1 không nhất thiết làm TWDT báo lỗi, và firmware cần tự đo thời gian thực hiện vòng lặp, ví dụ qua chỉ số loop_max_ms được ghi trong log, để phát hiện hiện tượng này.

2.4.5 Áp dụng trong đề tài

Trong lõi Arduino-ESP32, hàm setup() và loop() chạy bên trong một tác vụ FreeRTOS tên là loopTask, được tạo với mức ưu tiên 1 trên lõi ARDUINO_RUNNING_CORE (Espressif Systems, 2024). Với ESP32 hai lõi, giá trị mặc định của ARDUINO_RUNNING_CORE là core 1, còn tác vụ WiFi và tác vụ TCP/IP của thư viện lwIP (lightweight IP) được gắn vào core 0. Các giá trị này đã được kiểm tra trong tệp cores/esp32/main.cpp, tệp Kconfig.projbuild và cấu hình sdkconfig biên dịch sẵn của lõi Arduino-ESP32 phiên bản 2.0.17 (Espressif Systems,

2024), là phiên bản đề tài sử dụng. Sự phân bổ tác vụ trên hai lõi được minh họa ở Hình 2.1.



Hình 2.1. Phân bổ tác vụ trên hai lõi ESP32 trong lõi Arduino-ESP32

Thư viện painlessMesh xử lý mạng thông qua hàm mesh.update() được gọi trong loop(). Bên trong hàm này, thư viện dùng một bộ lập lịch hợp tác (TaskScheduler) chạy ngay trong loopTask, nghĩa là các công việc của mesh chỉ được thực hiện khi loop() quay lại gọi mesh.update(). Như vậy, mọi đoạn mã chặn lâu trong loop(), chẳng hạn ghi một lượng lớn dữ liệu ra UART ở chế độ chặn, đều làm mesh.update() bị gọi thưa đi, dù tác vụ WiFi trên core 0 vẫn tiếp tục nhận gói.

Chương 4 ghi nhận một ví dụ thực tế của cơ chế này. Ở chu kỳ chụp 300 ms với ba Leaf, mỗi Leaf gửi khoảng 3,3 khung/giây và Root phải gửi khoảng 47 KB/s ảnh qua UART 921600 bps, trong khi giới hạn lý thuyết của liên kết này khoảng 92 KB/s. Root dành gần một nửa thời gian để ghi UART ở chế độ chặn, nên mesh.update() bị gọi thưa, dữ liệu dồn lại trong các kết nối TCP của painlessMesh và toàn mạng ngừng truyền dữ liệu.

Hướng khắc phục mà đề tài áp dụng là ghi UART không chặn. Root đặt một bộ đệm phát cho UART; khi bộ đệm còn đủ chỗ, lệnh ghi chỉ sao chép dữ liệu vào bộ

đệm rồi trả về ngay, còn việc truyền từng byte ra chân phát TX (Transmit) do driver UART thực hiện, nên loop() không phải chờ. Khi bộ đệm sắp đầy, Root bỏ khung ảnh thay vì chờ, như trình bày ở mục 3.4.1. Kết quả ban đầu của cơ chế này được trình bày ở Chương 4.

2.5 Nền tảng phần cứng

Hệ thống sử dụng hai loại board dựa trên vi điều khiển ESP32 (Espressif Systems, n.d.-d): board ESP32-CAM cho các Leaf và board ESP32 DevKitC cho Root. Mục này trình bày các đặc điểm của từng loại board có liên quan trực tiếp tới thiết kế hệ thống. Đặc biệt, dung lượng bộ nhớ của từng board quyết định kích thước vùng đệm ảnh có thể cấp phát.

2.5.1 Module ESP32-CAM và camera OV3660

ESP32-CAM là module tích hợp vi điều khiển ESP32, bộ nhớ PSRAM (Pseudo-Static Random Access Memory) ngoài và khe cắm camera. Board dùng cho Leaf trong đề tài là loại ESP32-CAM-MB có sẵn đế nạp USB dùng chip CH340, đi kèm camera OV3660 thay cho camera OV2640 nêu trong yêu cầu ban đầu. Khi đưa board vào hoạt động, firmware xác nhận PSRAM có dung lượng khoảng 4 MB, đủ để chứa bộ đệm khung ảnh của camera. Board được cấu hình trong Arduino IDE là AI Thinker ESP32-CAM với bảng phân vùng Huge APP (3 MB No OTA/1 MB SPIFFS).

2.5.2 Board ESP32 DevKitC dùng cho Root

Root dùng board ESP32-WROOM-32D DevKitC có chip chuyển đổi USB–UART CP2102, cho phép cắm thẳng vào máy tính để middleware đọc dữ liệu UART mà không cần mạch nạp rời. Board này không có PSRAM, nên sau khi khởi tạo mesh chỉ còn khoảng 120 KB heap trống trong SRAM (Static Random Access Memory) nội. Đặc điểm này quyết định kích thước và số lượng vùng đệm ghép ảnh có thể cấp phát tại Root, như trình bày ở Chương 3.

2.6 Các chỉ số đánh giá hiệu năng mạng

Đề tài đánh giá hiệu năng mạng bằng năm chỉ số: thông lượng, số khung trên giây, tỉ lệ hủy khung, độ trễ và thời gian tự phục hồi. Các chỉ số được tính tại Root và hiển thị trên Web Dashboard theo các công thức từ (2.1) đến (2.5). Thông lượng là lượng dữ liệu ảnh nhận được thành công tại Root trong một đơn vị thời gian và được tính theo (2.1).

$$T = \frac{B}{\Delta t} \tag{2.1}$$

trong đó:

T là thông lượng (KB/s);

B là tổng dung lượng ảnh của các khung được ghép thành công tại Root trong khoảng quan sát (KB);

Δt là độ dài khoảng thời gian quan sát (s).

Số khung trên giây (Frames Per Second – FPS) là số ảnh được ghép hoàn chỉnh mỗi giây. Chỉ số này được tính cho toàn mạng, gọi là FPS tổng, hoặc cho từng Leaf, và phản ánh trực tiếp tốc độ cập nhật ảnh trên Web Dashboard. FPS được tính theo (2.2).

$$\mathrm{FPS} = \frac{N_{\mathrm{ok}}}{\Delta t} \tag{2.2}$$

trong đó:

FPS là số khung trên giây (khung/giây);

N_{ok} là số khung được ghép thành công trong khoảng quan sát (khung).

Tỉ lệ hủy khung là tỉ lệ khung không ghép được do thiếu chunk hoặc hết thời gian chờ. Vì hệ thống chưa có cơ chế gửi lại, một khung chỉ cần mất một chunk là bị hủy, nên chỉ số này phản ánh cả việc mất gói trên mạng mesh lẫn tình trạng quá tải tại Root. Tỉ lệ hủy khung được tính theo (2.3).

$$R_{\mathrm{h}} = \frac{N_{\mathrm{h}}}{N_{\mathrm{ok}} + N_{\mathrm{h}}} \times 100 \tag{2.3}$$

trong đó:

R_{h} là tỉ lệ hủy khung (%);

N_{h} là số khung bị hủy do thiếu chunk hoặc hết thời gian chờ trong khoảng quan sát (khung).

Độ trễ của một khung ảnh là khoảng thời gian từ lúc Leaf chụp ảnh đến lúc Root ghép xong ảnh. Hai mốc thời gian này được lấy theo đồng hồ chung của mạng mesh do painlessMesh duy trì, nên không cần đồng bộ thêm giữa Leaf và Root. Độ trễ được tính theo (2.4).

$$d = t_{\text{ghép}} - t_{\text{chụp}} \tag{2.4}$$

trong đó:

- d là độ trễ của khung ảnh (ms);
- $t_{\text{chụp}}$ là thời điểm Leaf chụp ảnh, theo đồng hồ chung của mạng mesh (ms);
- $t_{\text{ghép}}$ là thời điểm Root ghép xong ảnh, theo đồng hồ chung của mạng mesh (ms).

Thời gian tự phục hồi là khoảng thời gian từ lúc một node mất kết nối đến lúc dữ liệu từ node đó tới Root trở lại. Cách tính này dựa trên luồng dữ liệu thực tế thay vì sự kiện thay đổi topology của thư viện, nên phản ánh thời điểm node thật sự gửi được ảnh trở lại. Thời gian tự phục hồi được tính theo (2.5).

$$T_{\text{ph}} = t_{\text{nối}} - t_{\text{mất}} \tag{2.5}$$

trong đó:

- T_{ph} là thời gian tự phục hồi (ms);
- $t_{\text{mất}}$ là thời điểm Root nhận chunk cuối cùng của node trước khi mất kết nối (ms);
- $t_{\text{nối}}$ là thời điểm Root nhận lại chunk đầu tiên từ node đó (ms).

2.7 Kết luận chương

Chương 2 đã trình bày các cơ sở lý thuyết của đề tài, từ mạng mesh không dây, các nhóm giao thức định tuyến và mạng ad-hoc cho thiết bị bay đến hai giải pháp mesh chạy trên ESP32. Đề tài chọn painlessMesh vì thư viện này chạy trực tiếp trên ESP32, đồng thời lưu ý các giới hạn về kích thước bản tin, cơ chế broadcast và thời gian phát hiện mất kết nối. Phần FreeRTOS giải thích vì sao việc chặn loop() làm mesh.update() bị gọi thưa, còn các đặc điểm phần cứng và năm chỉ số đánh giá là đầu vào cho thiết kế ở Chương 3.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

Dựa trên cơ sở lý thuyết ở Chương 2, chương này trình bày thiết kế của hệ thống theo thứ tự từ tổng quát đến chi tiết. Phần đầu chương mô tả sơ đồ khối tổng thể gồm ba lớp, sau đó lần lượt đi vào phần cứng các node, firmware trên mạng mesh và phần mềm trên máy tính. Mỗi linh kiện và mỗi tham số được giới thiệu cùng với khối sử dụng nó.

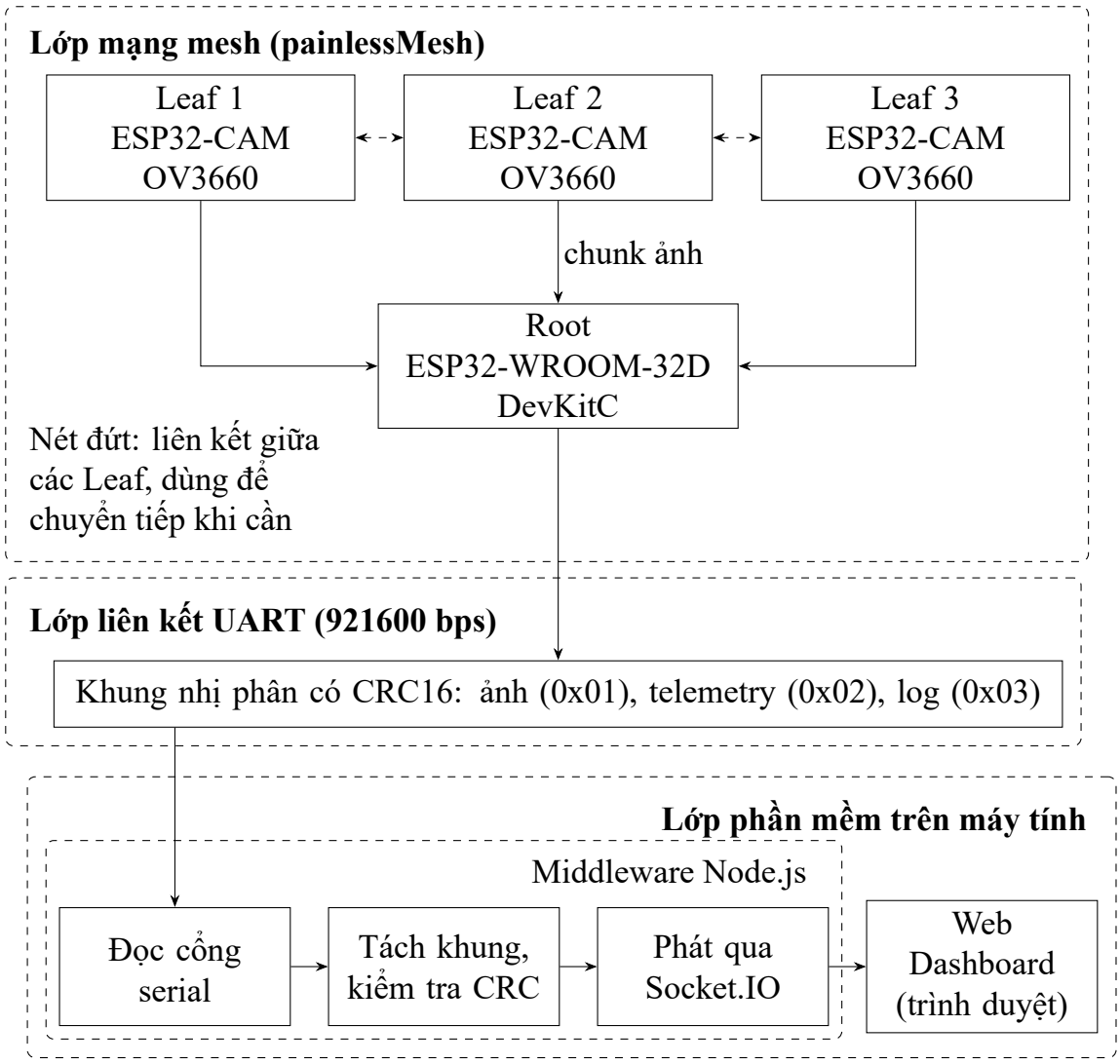
3.1 Kiến trúc tổng thể

Hệ thống được tổ chức thành ba lớp như Hình 3.1. Lớp mạng mesh gồm ba Leaf mang camera và một Root; mỗi Leaf chụp ảnh JPEG, chia ảnh thành các chunk và gửi về Root qua mạng mesh, và khi cần, một Leaf khác có thể đóng vai trò node chuyển tiếp. Lớp liên kết UART là đường truyền tốc độ 921600 bps giữa Root và máy tính, trên đó Root gửi ảnh đã ghép, bản tin trạng thái (telemetry) và log theo một giao thức khung nhị phân có kiểm tra CRC. Lớp phần mềm trên máy tính gồm middleware Node.js đọc cổng serial, tách khung và phát dữ liệu qua Socket.IO tới Web Dashboard chạy trên trình duyệt.

Cách chia lớp này tách việc truyền vô tuyến khỏi việc hiển thị. Root chỉ gửi lên máy tính những khung ảnh đã ghép hoàn chỉnh, nên middleware và dashboard không phải xử lý chunk hay mất gói trên mạng mesh. Các mục tiếp theo trình bày thiết kế của từng lớp theo đúng thứ tự trong Hình 3.1.

3.2 Thiết kế phần cứng

Phần cứng của hệ thống gồm một Root nối với máy tính và ba Leaf mang camera chạy pin độc lập. Vai trò, định danh (Identifier – ID) và phần cứng của từng node được tổng hợp trong Bảng 3.1. Các mục con tiếp theo trình bày cấu hình của từng loại node, trong đó khối nguồn được trình bày cùng với Leaf vì chỉ Leaf chạy bằng pin.



Hình 3.1. Sơ đồ khối tổng thể của hệ thống

Bảng 3.1. Vai trò và phần cứng các node

Vai trò	Node ID	Phần cứng và ghi chú
Root	2846999225	ESP32-WROOM-32D DevKitC, không có PSRAM, chip USB–UART CP2102, UART 921600 bps
Leaf	1207369673	ESP32-CAM-MB OV3660, pin 18650 và module tăng áp; từng gặp sự cố nguồn khi cấp qua USB
Leaf	1207433521	ESP32-CAM-MB OV3660, pin 18650 và module tăng áp
Leaf	1207449329	ESP32-CAM-MB OV3660, pin 18650 và module tăng áp

3.2.1 Node Root

Root dùng board ESP32-WROOM-32D DevKitC, được cấp nguồn và giao tiếp với máy tính qua cổng USB thông qua chip chuyển đổi USB–UART CP2102. Nhờ chip này, middleware đọc trực tiếp luồng dữ liệu UART của Root như một cổng COM (Communication port) mà không cần mạch nạp rời. Root không xử lý camera nên không cần PSRAM; firmware được điều chỉnh để mọi vùng đệm nằm trong SRAM nội, và giới hạn khoảng 120 KB heap trống của board quyết định số vùng đệm ghép ảnh ở mục 3.3.3.

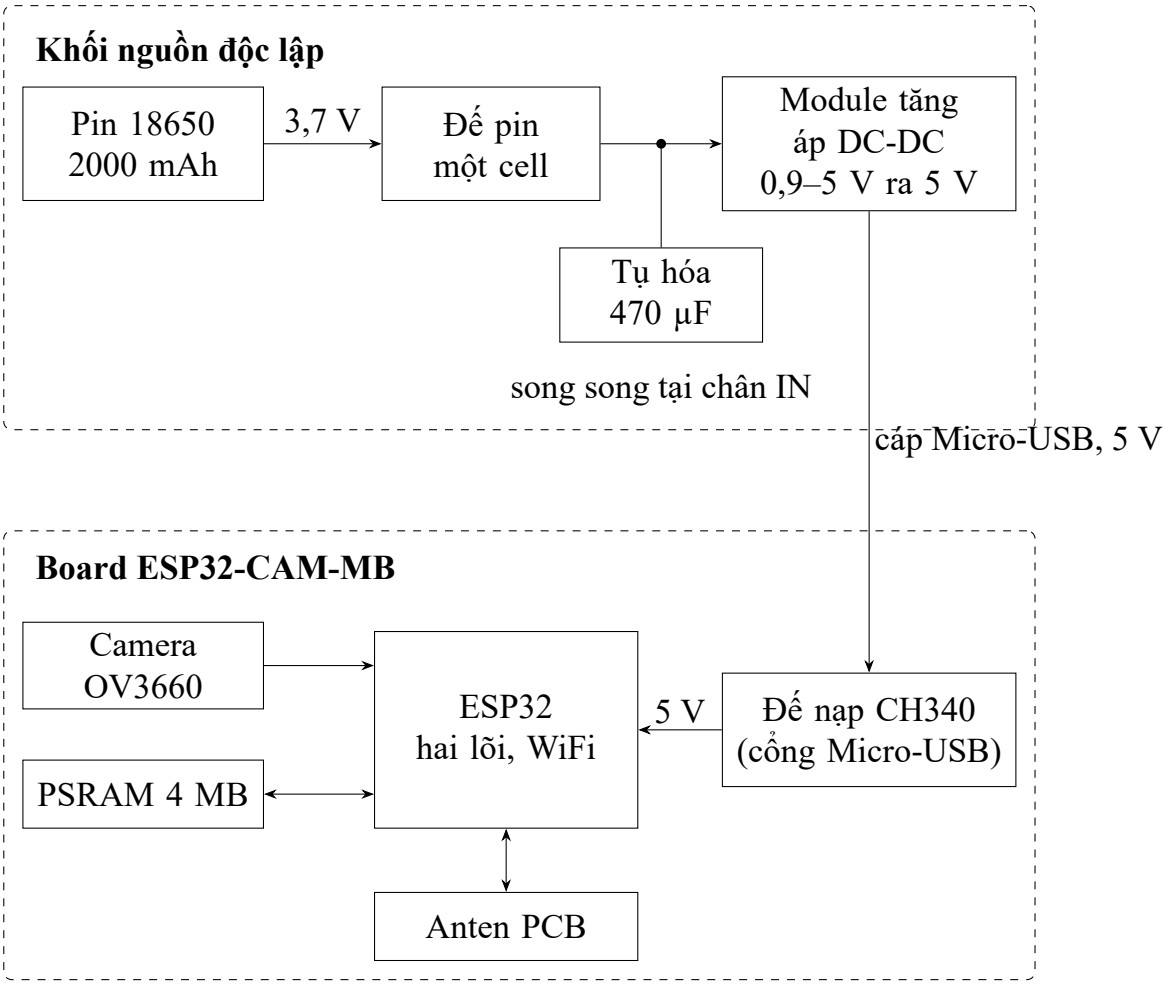
3.2.2 Node Leaf và khối nguồn

Mỗi Leaf gồm board ESP32-CAM-MB với camera OV3660 và một khối nguồn độc lập để có thể tách node ra xa máy tính. Khối nguồn gồm một pin 18650 loại 2000 mAh, đế pin một cell và module tăng áp một chiều DC-DC (Direct Current) nhận đầu vào 0,9–5 V, đầu ra 5 V qua cổng USB-A. Đề tài sử dụng mạch tăng áp thay vì mạch hạ áp LM2596 nêu trong tài liệu ban đầu, vì một cell 18650 chỉ có điện áp danh định khoảng 3,7 V, thấp hơn mức 5 V cần cấp cho board.

Do đầu ra của module là cổng USB-A, board được nối với khối nguồn bằng cáp Micro-USB thông thường. Điểm hàn duy nhất là dây từ đế pin vào chân IN của module, kèm một tụ hóa 470 µF hàn song song tại chân IN để giảm sụt áp khi ESP32 phát WiFi với dòng đỉnh lớn. Trước khi cắm vào board, điện áp đầu ra của module được kiểm tra bằng đồng hồ vạn năng theo quy trình hàn tám bước đã soạn. Sơ đồ khối phần cứng của một Leaf được trình bày ở Hình 3.2.

3.2.3 Tổng hợp linh kiện

Bảng 3.2 tổng hợp các linh kiện chính đã giới thiệu ở hai mục trên cùng số lượng thực tế đang dùng. Số pin, đế pin và module tăng áp nhiều hơn số Leaf để có linh kiện dự phòng khi một khối nguồn bị hỏng. Các phụ kiện như tụ hóa và dây cắm được mua một lần và dùng chung cho tất cả các node.



Hình 3.2. Sơ đồ khối phần cứng của một Leaf

Bảng 3.2. Danh sách linh kiện chính

Linh kiện	Số lượng	Mục đích
ESP32-WROOM-32D DevKitC (CP2102)	1	Root
ESP32-CAM-MB OV3660 (đế nạp CH340)	3	Leaf
Pin 18650 2000 mAh	4	Nguồn Leaf (1 dự phòng)
Đế pin 18650 một cell	5	Nguồn Leaf
Module tăng áp DC-DC 0,9–5 V ra 5 V USB	5	Nguồn Leaf (dự phòng)
Tụ hóa 470 µF 16 V	10	Lọc nguồn tại chân IN của module tăng áp
Dây cắm test board đực – cái 20 cm	3 bẹ	Đấu nối thử nghiệm

3.3 Thiết kế firmware trên mạng mesh

Firmware trên mạng mesh đảm nhận việc chụp, phân mảnh, truyền và ghép ảnh. Mục này trình bày cấu trúc gói chunk dùng chung cho cả hai loại node, sau đó là firmware của Leaf, firmware của Root và các tham số cấu hình chính. Firmware của cả hai loại node chạy trong tác vụ loopTask của lõi Arduino-ESP32 đã trình bày ở mục 2.4.

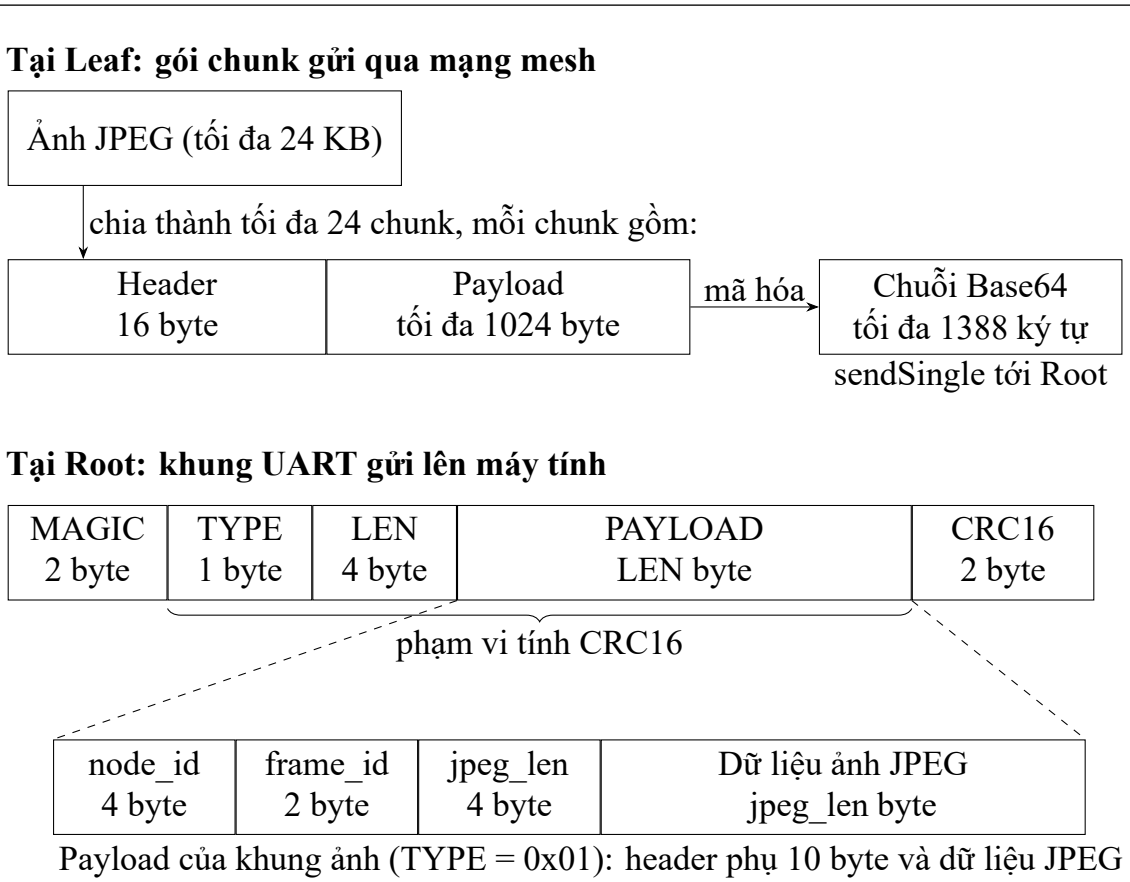
3.3.1 Cơ chế phân mảnh và cấu trúc gói chunk

Kích thước một ảnh JPEG lớn hơn nhiều so với giới hạn khoảng 1400 byte của một bản tin painlessMesh, nên ảnh phải được chia thành các mảnh dữ liệu (chunk). Mỗi chunk mang tối đa 1024 byte dữ liệu ảnh và một header 16 byte như Bảng 3.3; kích thước 1024 byte được chọn gần với giới hạn bản tin vì giảm kích thước chunk không làm giảm đụng độ trên kênh dùng chung, như đã phân tích ở mục 2.3. Header cho phép Root biết chunk thuộc node nào, khung nào, ở vị trí nào và khung có tổng bao nhiêu chunk. Trường capture_us là mốc thời gian chụp theo đồng hồ chung của mesh, được bổ sung ngày 30/09 để đo độ trễ; trước đó header chỉ có 12 byte.

Bảng 3.3. Cấu trúc header chunk 16 byte

Offset	Trường	Kiểu	Số byte
0	node_id	uint32_t	4
4	frame_id	uint16_t	2
6	chunk_index	uint16_t	2
8	total_chunks	uint16_t	2
10	payload_len	uint16_t	2
12	capture_us	uint32_t	4
	Tổng		16

Vì painlessMesh truyền bản tin dạng chuỗi ký tự, toàn bộ header và payload được mã hóa Base64, nên một gói thực tế dài tối đa 1388 ký tự. Phía Root, ảnh sau khi ghép được đóng vào khung UART với một header phụ riêng, trình bày ở mục 3.4.1. Quan hệ giữa gói chunk trên mạng mesh và khung truyền qua UART được minh họa ở Hình 3.3.



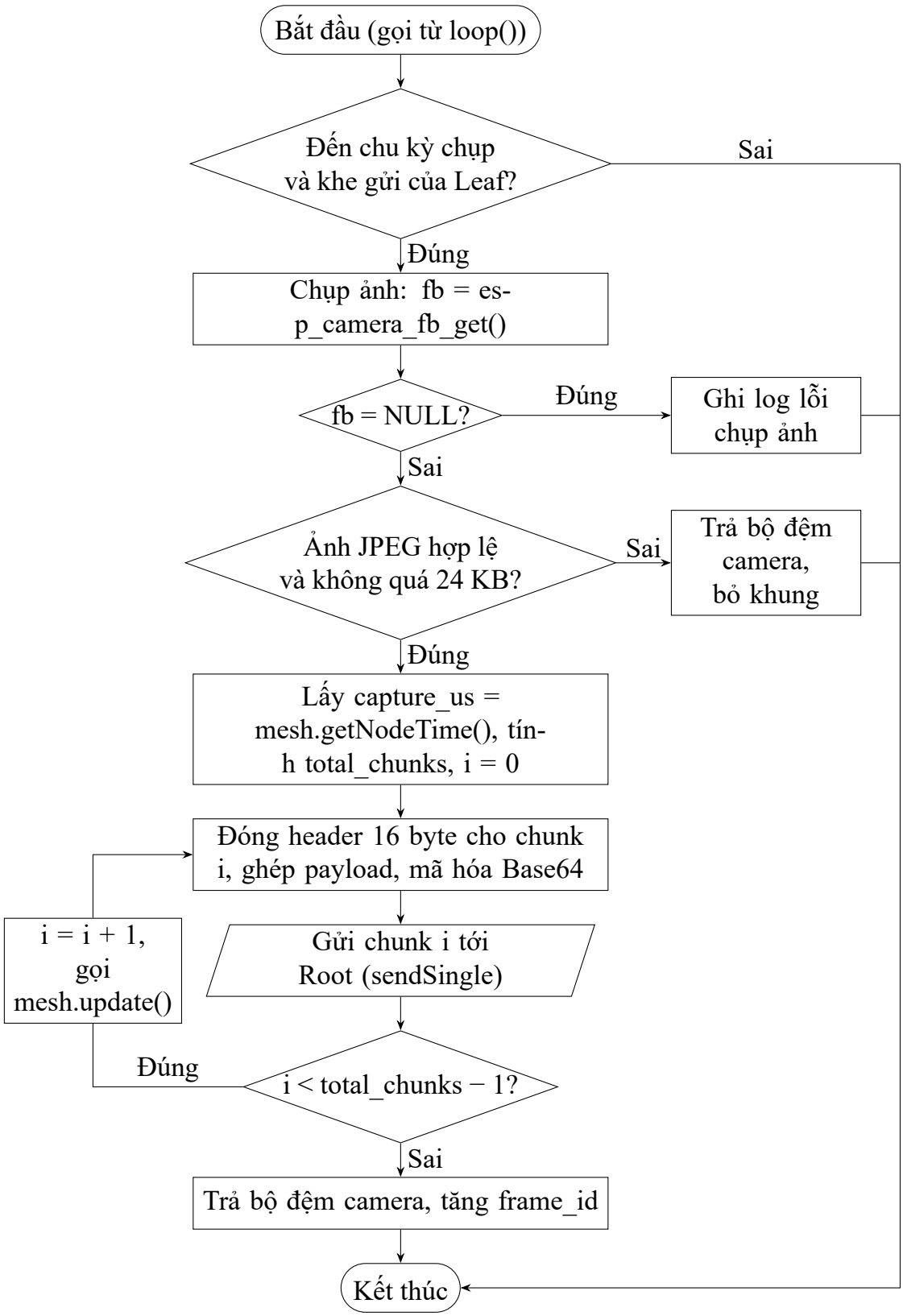
Hình 3.3. Cấu trúc gói chunk trên mạng mesh và khung UART

3.3.2 Firmware node Leaf

Lưu đồ chụp và gửi ảnh của Leaf được trình bày ở Hình 3.4. Theo chu kỳ CAPTURE_INTERVAL_MS, Leaf chụp một ảnh, kiểm tra ảnh hợp lệ, lấy mốc thời gian bằng mesh.getNodeTime(), chia ảnh thành các chunk và gửi lần lượt. Leaf tự bỏ những khung lớn hơn 24 KB để khớp với kích thước vùng đệm ghép ảnh của Root.

Ở phiên bản đầu, Leaf gửi bằng sendBroadcast, khiến mỗi chunk bị phát tới mọi node, kể cả các Leaf khác vốn không ghép ảnh. Phiên bản hiện tại để Root phát beacon "#ROOT" mỗi 2 giây; Leaf học định danh Root từ beacon này và gửi chunk bằng sendSingle thẳng tới Root. Ký tự # được chọn vì không thuộc bảng mã Base64 nên bản tin điều khiển không bị nhầm với chunk.

Để giảm tranh chấp kênh khi nhiều Leaf cùng gửi, firmware áp dụng cơ chế gửi lệch pha (staggering) dựa trên đồng hồ chung của mesh. Chu kỳ chụp 600 ms được chia thành ba khe 200 ms, mỗi Leaf được gán một khe cố định theo node ID;



Hình 3.4. Lưu đồ chụp và gửi ảnh tại Leaf

node không có trong bảng gán khe được xếp khe bằng phép chia lấy dư node ID cho số khe. Nhờ đó, tại mỗi thời điểm chỉ có một Leaf gửi chuỗi chunk của mình.

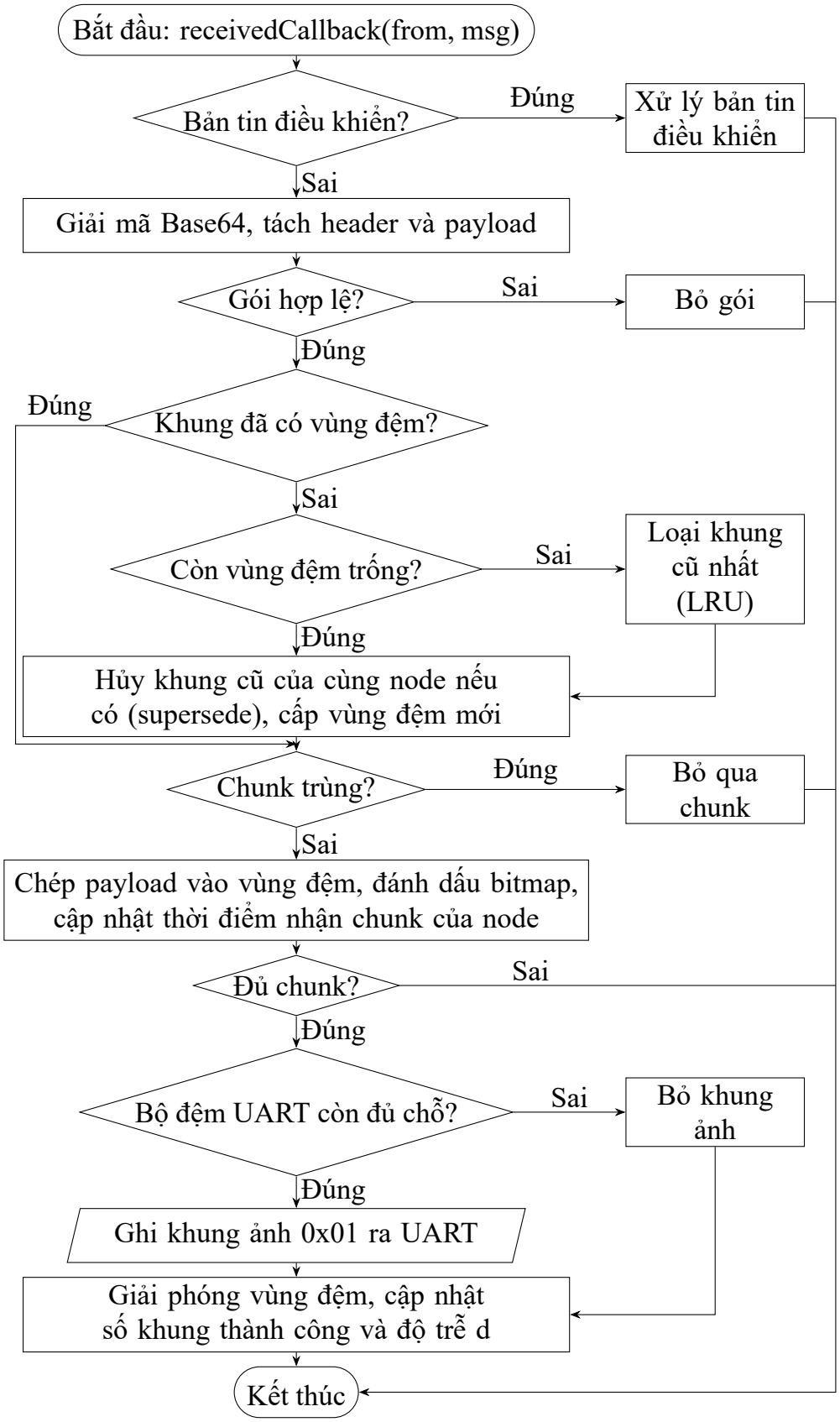
Leaf còn có cơ chế watchdog ở mức ứng dụng. Nếu mất beacon của Root quá 8 giây, Leaf quên định danh Root để học lại; nếu mất quá 30 giây, Leaf khởi động lại board. Ngoài ra, mỗi 2 giây Leaf gửi bản tin #NSTAT về Root chứa RSSI, heap, phiên bản firmware, nguyên nhân khởi động lại gần nhất và thời gian hoạt động.

3.3.3 Firmware node Root

Root nhận chunk, giải mã Base64 và ghép ảnh theo lưu đồ ở Hình 3.5. Mỗi khung đang ghép được giữ trong một vùng đệm ghép ảnh (slot) có bitmap đánh dấu các chunk đã nhận và mốc thời gian để hủy khung quá 2 giây chưa đủ chunk. Root có bốn vùng đệm, mỗi vùng 24 KB, tổng cộng 96 KB SRAM; số vùng đệm phải lớn hơn số Leaf gửi đồng thời để chunk của Leaf này không chiếm mất vùng đệm của khung đang ghép dở của Leaf khác.

Khi chunk của một khung mới từ cùng một node tới, Root hủy ngay khung cũ chưa xong của node đó (cơ chế thay thế, supersede), vì Leaf gửi tuần tự nên khung cũ không thể hoàn thành nữa. Khi hết vùng đệm trống, khung cũ nhất bị loại theo kiểu LRU (Least Recently Used). Khi một khung đủ chunk, Root đóng khung ảnh và ghi ra UART, rồi giải phóng vùng đệm cho khung tiếp theo.

Ngoài chức năng ghép ảnh, Root gửi telemetry mỗi 2 giây gồm số khung ghép thành công và bị hủy, số chunk, heap trống, danh sách node, cây topology lấy từ thư viện, số hop và RSSI từng liên kết, độ trễ trung bình, nhỏ nhất, lớn nhất theo từng node và thời gian tự phục hồi. Root theo dõi thời điểm nhận chunk hợp lệ gần nhất của từng node trong một mảng tĩnh tám phần tử; node có trong danh sách mesh nhưng không gửi chunk quá 10 giây được đánh dấu im lặng để phát hiện node vẫn được đếm nhưng không còn gửi ảnh. Nếu heap xuống dưới 40 KB liên tục quá 10 giây, Root ghi log và tự khởi động lại.



Hình 3.5. Lưu đồ nhận và ghép ảnh tại Root

3.3.4 Các tham số cấu hình chính

Các tham số quan trọng của firmware được đặt trong tệp mesh_config.h của từng node và được tổng hợp ở Bảng 3.4. Nhóm tham số đầu quy định kích thước chunk, khung và vùng đệm ghép ảnh, nhóm tiếp theo quy định chu kỳ chụp, beacon và gửi lệch pha. Nhóm cuối gồm các ngưỡng giám sát của watchdog, node im lặng, bảo vệ heap và bộ đệm phát UART.

Bảng 3.4. Các tham số cấu hình chính của firmware

Tham số	Giá trị	Ý nghĩa
Payload tối đa mỗi chunk	1024 byte	Dữ liệu ảnh trong một chunk
MAX_CONCURRENT_ FRAMES	4	Số khung ghép song song tại Root
MAX_FRAME_SIZE	24576 byte	Kích thước tối đa một khung
MAX_CHUNKS_PER_ FRAME	24	Số chunk tối đa một khung
REASSEMBLY_TIMEOUT_ MS	2000 ms	Thời gian chờ ghép một khung
CAPTURE_INTERVAL_MS	600 ms	Chu kỳ chụp của Leaf (trước là 300 ms)
ROOT_BEACON_ INTERVAL_MS	2000 ms	Chu kỳ phát beacon #ROOT
ROOT_LOST_RESET_MS / RESTART_MS	8000 / 30000 ms	Watchdog Leaf khi mất beacon
STAGGER_SLOTS	3	Số khe gửi lệch pha
SILENT_NODE_MS	10000 ms	Ngưỡng đánh dấu node im lặng
HEAP_CRITICAL_BYTES / MS	40960 byte / 10000 ms	Ngưỡng bảo vệ heap tại Root
UART_TX_BUFFER_SIZE	8192 byte	Bộ đệm phát UART tại Root

3.4 Thiết kế giao tiếp và phần mềm trên máy tính

Mục này trình bày lớp liên kết UART và lớp phần mềm trên máy tính trong Hình 3.1. Nội dung gồm giao thức truyền dữ liệu giữa Root và máy tính, middleware xử lý dữ liệu và giao diện giám sát Web Dashboard. Ba thành phần này được thiết kế để luồng ảnh hiển thị trên trình duyệt không bị gián đoạn khi luồng byte từ cổng serial có lỗi.

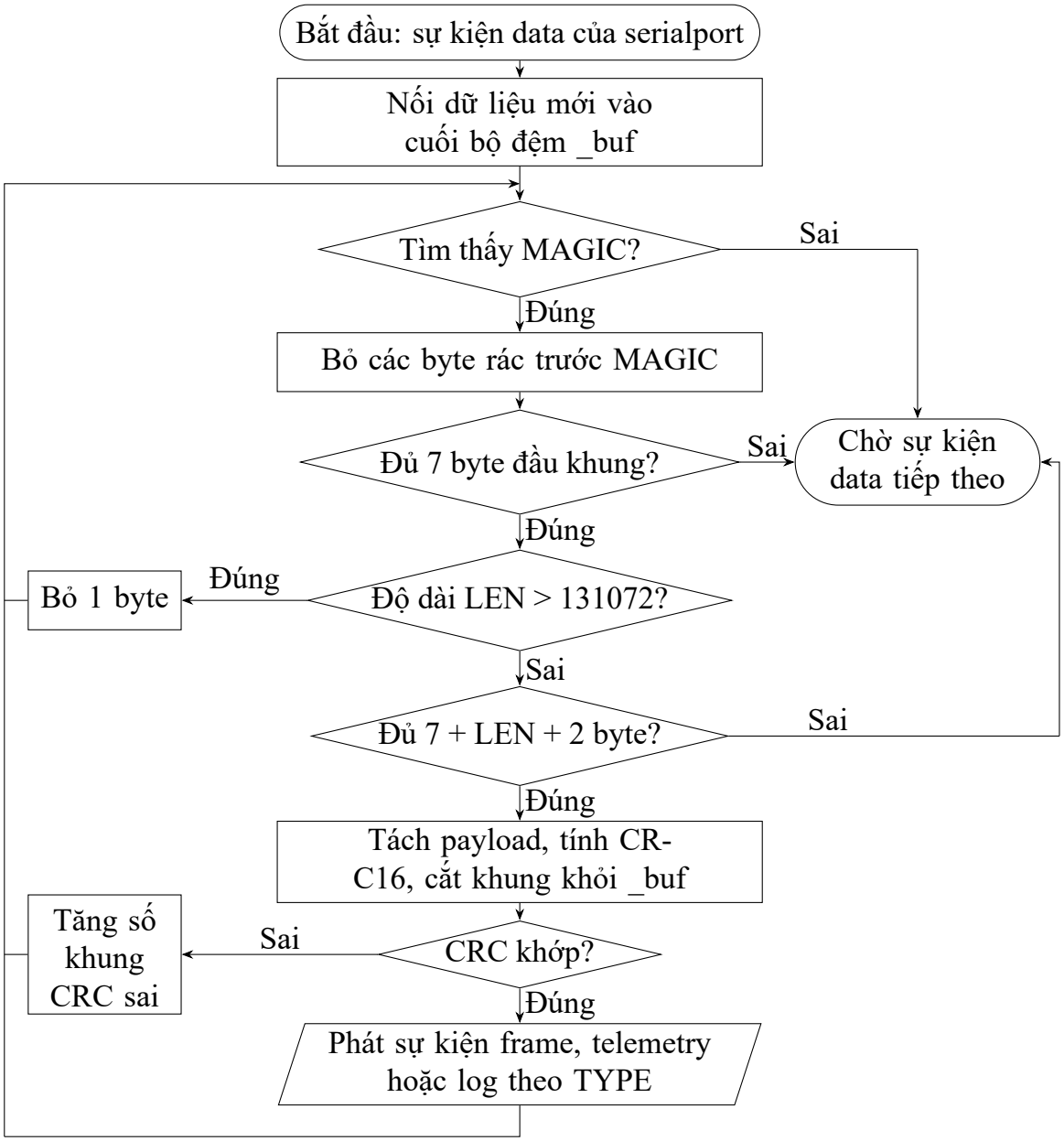
3.4.1 Giao thức truyền dữ liệu qua UART

Root gửi dữ liệu lên máy tính qua UART 921600 bps theo giao thức khung nhị phân được đặc tả trong tệp UART_PROTOCOL.md, có cấu trúc như Bảng 3.5. Khung bắt đầu bằng hai byte đồng bộ AA 55, tiếp theo là một byte kiểu, bốn byte độ dài, phần dữ liệu và hai byte CRC16/CCITT-FALSE tính trên phần từ byte kiểu đến hết dữ liệu. Có ba kiểu khung là ảnh đã ghép (0x01), telemetry dạng JSON (0x02) và log (0x03); khung ảnh có thêm header phụ 10 byte gồm node_id, frame_id và độ dài ảnh JPEG.

Bảng 3.5. Cấu trúc khung UART

Trường	Số byte	Mô tả
MAGIC	2	AA 55, đánh dấu đầu khung
TYPE	1	0x01 ảnh, 0x02 telemetry, 0x03 log
LEN	4	Độ dài payload (uint32, little-endian)
PAYLOAD	LEN	Dữ liệu theo kiểu khung
CRC16	2	CRC16/CCITT-FALSE trên TYPE..PAYLOAD

Để việc ghi UART không chặn vòng lặp chính, Root đặt bộ đệm phát 8192 byte trước khi khởi tạo cổng serial và bỏ khung ảnh nếu dung lượng còn trống của bộ đệm nhỏ hơn một nửa; telemetry và log không bao giờ bị bỏ. Số khung ảnh bị bỏ tại UART được đếm và đưa lên dashboard. Mức log của painlessMesh được hạ xuống chỉ báo lỗi, vì log văn bản chèn vào luồng nhị phân khiến middleware phải bỏ qua nhiều byte để tìm lại đầu khung.

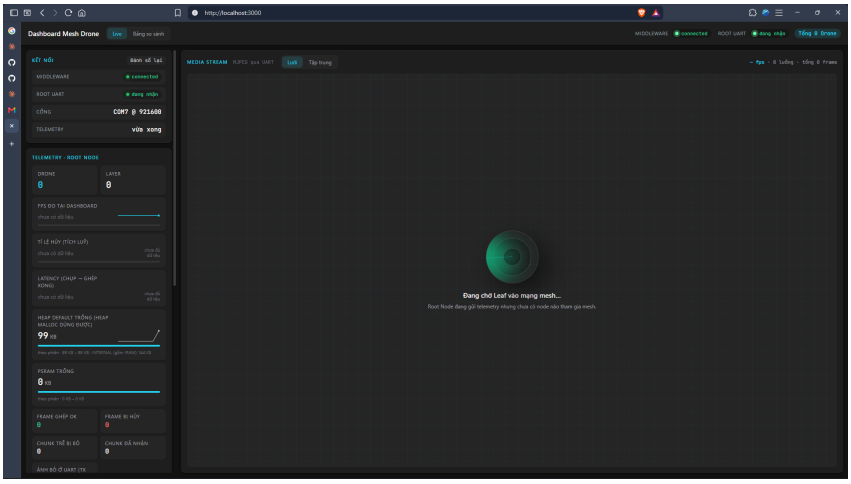


Hình 3.6. Lưu đồ tách khung tại middleware

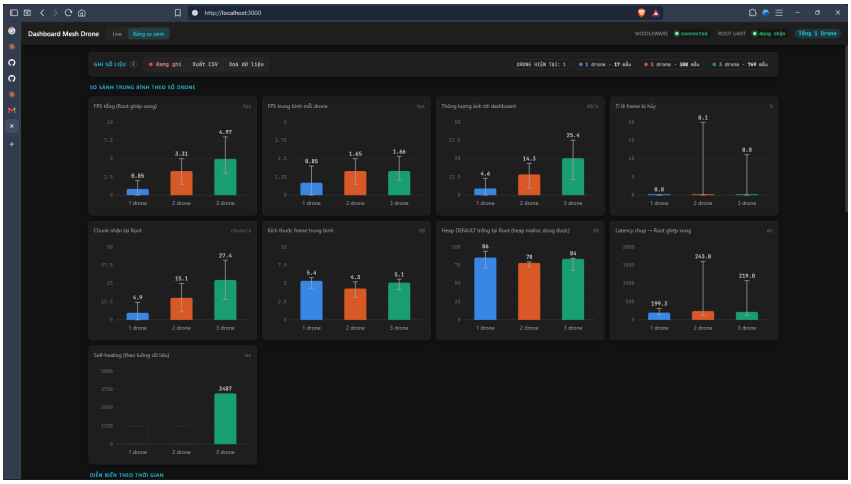
3.4.2 Middleware trên máy tính

Middleware được viết bằng Node.js, đọc cổng serial bằng thư viện serialport và phát dữ liệu tới trình duyệt bằng Socket.IO qua năm sự kiện: connect, disconnect, status, telemetry và frame. Bộ tách khung tìm hai byte MAGIC, kiểm tra độ dài (chấp nhận tối đa 131072 byte) và CRC trước khi phát dữ liệu, theo lưu đồ ở Hình 3.6. Khi gặp byte rác, độ dài vô lý hoặc CRC sai, bộ tách khung chỉ bỏ phần dữ liệu lỗi và tiếp tục quét, nên một khung hỏng không làm dừng cả luồng. Middleware không ghép ảnh

vì Root đã gửi ảnh hoàn chỉnh. Bộ kiểm thử test_protocol.js đã kiểm tra bảy kịch bản, gồm dữ liệu bị cắt vụn, có byte rác xen giữa và CRC sai, kết quả đạt 7/7.



(a) Tab Live



(b) Tab So sánh

Hình 3.7. Giao diện Web Dashboard: (a) tab Live; (b) tab So sánh

3.4.3 Web Dashboard

Dashboard là trang web sử dụng React 18 và Tailwind CSS, nhận dữ liệu thời gian thực từ middleware và gồm hai tab chính như Hình 3.7. Tab Live ở Hình 3.7a hiển thị ảnh từ từng Leaf, trạng thái kết nối, cây topology vẽ bằng đồ họa vector SVG (Scalable Vector Graphics) với màu cạnh theo RSSI, số hop và độ trễ. Tab So sánh ở Hình 3.7b tính tốc độ thay đổi của số khung thành công, số khung hủy và số chunk từ telemetry, sau đó nhóm các mẫu theo số node để so sánh FPS tổng, FPS mỗi Leaf,

thông lượng, tỉ lệ hủy khung, số chunk mỗi giây, kích thước khung trung bình, heap trống tại Root và thời gian tự phục hồi.

Số liệu trên tab So sánh có thể xuất ra tệp CSV để phân tích. Nguyên tắc thiết kế của dashboard là không sử dụng dữ liệu giả lập: khi chưa có dữ liệu, dashboard hiển thị trạng thái chưa có dữ liệu. Các số liệu ở Chương 4 được ghi bằng tab So sánh và xuất ra tệp CSV theo cách này.

3.5 Kết luận chương

Chương 3 đã trình bày thiết kế hệ thống từ sơ đồ khối ba lớp tới từng khối phần cứng, firmware và phần mềm. Ở lớp mạng mesh, ảnh JPEG được chia thành các chunk tối đa 1024 byte kèm header 16 byte, được Leaf gửi thẳng tới Root bằng sendSingle theo khe lệch pha, rồi được Root ghép bằng bốn vùng đệm với cơ chế thay thế và loại bỏ theo LRU. Ảnh đã ghép được đóng khung nhị phân có CRC16 và truyền qua UART tới middleware để tách khung và hiển thị trên Web Dashboard. Chương 4 tiếp theo trình bày kết quả thử nghiệm của hệ thống đã thiết kế.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày môi trường và phương pháp thử nghiệm, quá trình đưa phần cứng vào hoạt động, kết quả đo với một và nhiều node Leaf, kết quả đo thời gian tự phục hồi và các cải tiến ổn định đã thực hiện.

4.1 Môi trường và phương pháp thử nghiệm

Các thử nghiệm được thực hiện với một Root nối máy tính chạy Windows và tối đa ba Leaf. Leaf được nạp firmware qua đế CH340 và chạy bằng pin khi đo. Số liệu được ghi bằng tab So sánh của dashboard và xuất ra các tệp CSV; mỗi đợt đo được thực hiện lần lượt với một, hai và ba Leaf để so sánh. Tải dữ liệu được thay đổi thông qua chu kỳ chụp của Leaf, gồm hai mức 300 ms và 600 ms.

4.2 Quá trình đưa phần cứng vào hoạt động

Trong quá trình bring-up, đề tài gặp và xử lý nhiều sự cố, tổng hợp ở Bảng 4.1. Đáng chú ý là lỗi của thư viện painlessMesh gốc khiến Leaf không tìm được Root, và hiện tượng tranh chấp sóng khi hai Leaf đặt sát nhau trên bàn. Sau khi xử lý, Leaf đầu tiên gửi ảnh liên tục với 1306 khung thành công và 6 khung bị hủy (khoảng 0,5 %), không có khung UART sai CRC.

4.3 Kết quả với một node Leaf

Ở chu kỳ chụp 600 ms, một Leaf đạt 1,67 ± 0,40 khung/giây, thông lượng 7,8 KB/s, 8,4 chunk/s, tỉ lệ hủy khung 0 %, kích thước khung trung bình khoảng 4,6 KB và heap trống tại Root ổn định ở 111–113 KB. Đây là mốc chuẩn cho một Leaf được dùng để so sánh với các trường hợp nhiều Leaf.

Bảng 4.1. Các sự cố khi bring-up và cách xử lý

Sự cố	Nguyên nhân	Cách xử lý
Root treo ngay khi khởi động	Firmware bắt buộc PSRAM, board Root không có PSRAM	Cấp phát vùng đệm trong SRAM nội, bỏ khối kiểm tra chặn cứng
Middleware bỏ qua nhiều byte	Log văn bản của painlessMesh chèn vào luồng UART nhị phân	Hạ mức log xuống chỉ báo lỗi
Leaf gửi chunk thất bại 100 %	Lỗi ngừng quét mạng của painlessMesh gốc	Chuyển sang fork Alteriom/ painlessMesh
Leaf rớt kết nối liên tục	Root cố kết nối WiFi nhà, tranh chấp radio với AP mesh	Tắt kết nối station của Root
Một Leaf thất bại, heap tụt dần	Cáp/cổng USB không đủ dòng đỉnh	Đổi cáp và cổng USB, sau đó dùng nguồn pin
Hai Leaf đổi cha liên tục	Hai radio đặt cách nhau vài cm tranh chấp kênh	Tách các Leaf ra xa nhau

4.4 Kết quả với nhiều node Leaf

Khi tăng số Leaf gửi đồng thời, hệ thống bộc lộ các giới hạn về vùng đệm ghép ảnh và tải xử lý tại Root. Mục này trình bày các kết quả theo trình tự thử nghiệm cùng các nguyên nhân đã được xác định.

4.4.1 Hai Leaf và lỗi một vùng đệm ghép

Lần thử đầu tiên với hai Leaf gửi đồng thời ngày 17/09 cho kết quả 111 khung thành công và 496 khung bị hủy, tỉ lệ hủy khoảng 82 %, so với 0,5 % khi chỉ có một Leaf. Khi kiểm tra lại mã nguồn, nguyên nhân được xác định là Root chỉ còn một slot ghép ảnh, do số slot đã bị hạ từ 4 xuống 1 để vừa SRAM của board không có PSRAM. Khi chunk của Leaf thứ hai tới, Root phải hủy khung đang ghép dở của Leaf thứ nhất để lấy slot, và ngược lại, nên hầu như không khung nào ghép xong. Để khắc phục, firmware được sửa thành bốn slot 24 KB, kết hợp gửi sendSingle thay cho sendBroadcast.

4.4.2 Ba Leaf chạy pin và hiện tượng đứng mạng

Sau khi sửa, ở chu kỳ chụp 300 ms, FPS tổng tăng gần tuyến tính từ khoảng 3,5 lên 7 và 10 khung/giây tương ứng với một, hai và ba Leaf, tỉ lệ hủy chỉ 0–5 %. Tuy nhiên với ba Leaf, toàn bộ mạng bị đứng sau 6–26 giây: FPS tổng về 0 trong hơn 38 giây trong khi số node Root báo vẫn là 3, heap Root giảm từ khoảng 90 KB xuống 57 KB và tiếp tục giảm. Hai Leaf thỉnh thoảng cũng gặp hiện tượng này. Ngoài ra còn có hiện tượng node ma: Root vẫn đếm một node nhưng node đó không gửi được ảnh.

Nguyên nhân được xác định là Root quá tải. Ở mức ba Leaf với khoảng 3,3 khung/giây mỗi Leaf, Root phải đẩy khoảng 47 KB/s ảnh qua UART 921600 bps, trong khi giới hạn lý thuyết của liên kết này khoảng 92 KB/s. Root dành gần một nửa thời gian để ghi UART ở chế độ chặn, mesh.update() bị gọi thưa và dữ liệu dồn lại trong các kết nối TCP của painlessMesh.

Khi giảm tải khoảng 40 % bằng cách tăng chu kỳ chụp lên 600 ms, ba Leaf không còn đứng trong suốt thời gian đo, điều này xác nhận chẩn đoán trên. Cái giá phải trả là FPS tổng giảm từ khoảng 10 xuống khoảng 5.

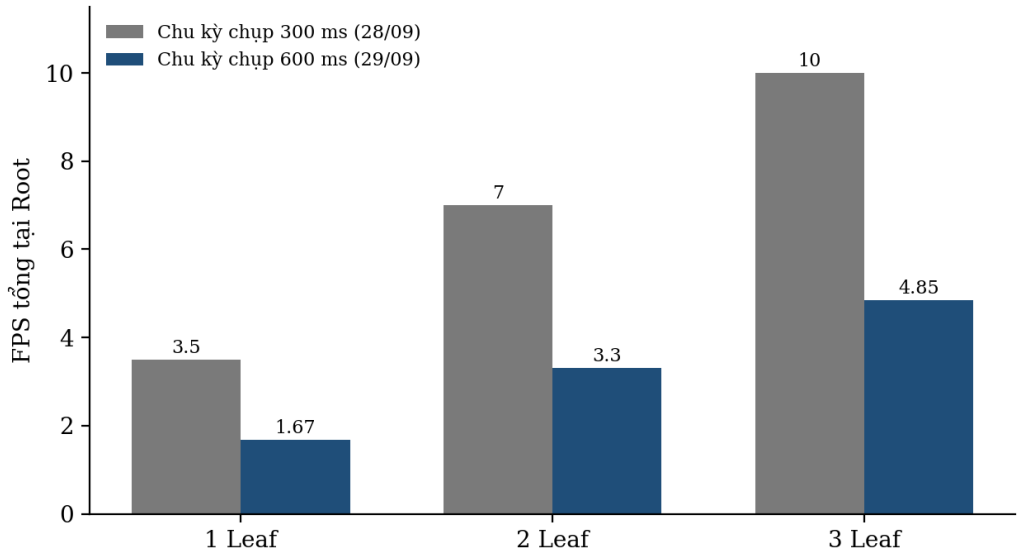
4.4.3 Kết quả ở chu kỳ chụp 600 ms

Bảng 4.2 tóm tắt kết quả ở chu kỳ 600 ms và Hình 4.1 so sánh FPS tổng giữa hai mức tải. Với ba Leaf, mạng chạy liên tục khoảng 2 phút, FPS tổng 4,85 ± 0,96, mỗi Leaf khoảng 1,6 khung/giây, 31 chunk/s tại Root, heap Root chủ yếu ở mức 95–101 KB. Có một đợt tỉ lệ hủy tăng lên 7–36 % trong khoảng 20 giây, heap thấp nhất 73 KB, sau đó mạng tự hồi phục.

Bảng 4.2. Kết quả thử nghiệm ở chu kỳ chụp 600 ms

Số Leaf	FPS tổng	Kết quả chính
1	1,67 ± 0,40	7,8 KB/s; 8,4 chunk/s; hủy 0 %; heap Root 111–113 KB
2	≈ 3,3	Chạy ổn khoảng 4 phút; một đợt hủy ngắn
3	4,85 ± 0,96 (≈ 5 ở phiên tối)	≈ 1,6 khung/giây mỗi Leaf; 31 chunk/s; heap Root 87–108 KB; chạy liền 3 phút không đứng

Trong phiên đo buổi tối ngày 29/09, diễn biến được ghi nhận như Bảng 4.3. Các Leaf rớt lần lượt chứ không đồng loạt, nên nguyên nhân không nằm ở Root.



Hình 4.1. So sánh FPS tổng theo số Leaf ở hai chu kỳ chụp (mức 300 ms là giá trị khi mạng còn chạy)

Bảng 4.3. Diễn biến phiên đo tối 29/09 (giờ UTC)

Thời gian	Số Leaf	Diễn biến
14:07:59 – 14:10:59	3	Chạy liền 3 phút, ≈ 5 FPS tổng, hủy gần 0 %, heap Root 87–108 KB
≈ 14:11	3 → 2	Leaf 1207369673 rớt, không quay lại (dừng ở khung #343)
14:12:01 – 14:16:15	2	Chạy ổn khoảng 4 phút, ≈ 3,3 FPS tổng, một đợt hủy ngắn
14:16:17 – 14:16:39	2 (thực tế 1)	Leaf 1207449329 ngừng gửi nhưng vẫn được đếm; heap Root 104 → 82 KB

4.5 Đo thời gian tự phục hồi

Root đo thời gian tự phục hồi từ sự kiện mất kết nối đến lần thay đổi topology kế tiếp. Các giá trị ghi được ngày 16/09 là 885, 10461 và 32528 ms; ngày 29/09 là 1243, 3944, 7691, 8823 và 19919 ms. Độ phân tán quá lớn nên các giá trị này chưa thể được sử dụng làm kết quả.

Nguyên nhân một phần nằm ở cách đo: trong thư viện, sự kiện thay đổi kết nối được gọi trước sự kiện mất kết nối, nên giá trị đo được là khoảng thời gian tới lần đổi topology bất kỳ, chưa chắc là lúc mạng thật sự nối lại. Ngoài ra, khi node mất nguồn

đột ngột, việc phát hiện phụ thuộc thời gian chờ khoảng 10 giây của painlessMesh. Ngày 30/09, firmware Root bổ sung cách đo mới dựa trên luồng dữ liệu: thời gian phục hồi được tính từ chunk cuối cùng nhận được của một node đến khi node đó gửi chunk trở lại, mỗi sự kiện được gửi lên dashboard đúng một lần và có thể xuất CSV cùng giá trị theo cách đo cũ. Cần đo self-healing ít nhất 5 lần theo quy trình đã thiết kế, tách riêng trường hợp đổi tuyến qua relay và trường hợp nối lại sau khi mất tín hiệu, ghi giá trị nhỏ nhất, lớn nhất và trung bình.

4.6 Các cải tiến ổn định và kết quả ban đầu

Ngày 30/09, firmware được bổ sung các cơ chế ổn định trên nhánh feature/on-dinh, bao gồm:

– Root không tự đếm chính nó và loại bỏ ID trùng trong danh sách node.

– Watchdog tại Leaf.

– Theo dõi node im lặng.

– Bảo vệ heap tại Root.

– Đo thời gian tự phục hồi theo luồng dữ liệu.

– Ghi UART không chặn và gửi lệch pha.

Phiên bản firmware tương ứng là 2026-09-30c cho Leaf và 2026-09-30d cho Root. Trong tệp đo test15-newfirmware.csv, số khung ảnh bị bỏ ở UART tăng từ 4 lên 20 trong khi FPS khoảng 3, cho thấy nhánh ghi không chặn đang hoạt động mà không bỏ hết ảnh; tuy nhiên tệp CSV không ghi phiên bản firmware nên chưa xác nhận được. Firmware cũng đã có đầy đủ phần đo độ trễ, cây topology và RSSI. Kết quả đo độ trễ và thông lượng theo 1, 2, 3 hop: không tìm thấy số liệu trong tài liệu hiện có, cần bổ sung sau khi bố trí chuỗi Root – Relay – Leaf và đo.

4.7 Kết luận chương

Chương 4 đã trình bày các kết quả thực nghiệm của hệ thống. Với một Leaf ở chu kỳ chụp 600 ms, hệ thống đạt khoảng 1,67 khung/giây và không có khung bị hủy. Khi tăng số Leaf, hai nút thắt chính được xác định là số vùng đệm ghép ảnh tại Root và tải ghi UART của Root; sau khi tăng lên bốn vùng đệm và giảm tải về chu kỳ chụp 600 ms, mạng ba Leaf chạy liên tục trong khoảng 2 phút với FPS tổng khoảng 4,85 khung/giây. Thời gian tự phục hồi đo được còn có độ phân tán lớn, và các phép đo độ trễ theo số hop cũng như dòng tiêu thụ vẫn cần được bổ sung. Chương 5 tiếp theo tổng kết các kết quả đạt được, các hạn chế và hướng phát triển của đề tài.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả mà đề tài đã đạt được dựa trên số liệu ở Chương 4 và nêu các hạn chế còn tồn tại. Phần cuối chương đề xuất hướng phát triển ngắn hạn và dài hạn cho hệ thống. Mỗi đề xuất được trình bày theo trình tự vấn đề, đề xuất, lý do và cách kiểm chứng.

5.1 Kết luận

Mục này tổng hợp các kết quả đạt được và các hạn chế của đề tài. Các kết quả và hạn chế chỉ được nêu trong phạm vi kết quả trình bày ở Chương 4. Những nội dung chưa được đo được ghi vào phần hạn chế và hướng phát triển.

5.1.1 Kết quả đạt được

Đề tài đã xây dựng một mạng WiFi Mesh gồm một Root và ba Leaf ESP32-CAM chạy pin độc lập, truyền ảnh JPEG từ nhiều Leaf về máy tính và hiển thị trên Web Dashboard theo thời gian thực. Chuỗi xử lý gồm chụp ảnh, chia chunk, truyền qua mạng mesh, ghép ảnh tại Root, đóng khung UART có CRC, tách khung ở middleware và hiển thị trên trình duyệt đã hoạt động trên phần cứng thật. Với một Leaf ở chu kỳ chụp 600 ms, hệ thống đạt 1,67 ± 0,40 khung/giây, thông lượng 7,8 KB/s và tỉ lệ hủy khung 0 %.

Qua thử nghiệm, đề tài xác định được hai nút thắt chính của hệ thống. Nút thắt thứ nhất là số vùng đệm ghép ảnh tại Root: với một vùng đệm, tỉ lệ hủy khung khoảng 82 % khi hai Leaf gửi đồng thời, và sau khi tăng lên bốn vùng đệm, tỉ lệ hủy ở chu kỳ chụp 300 ms còn 0–5 % trong khi FPS tổng tăng gần tuyến tính lên khoảng 3,5, 7 và 10 khung/giây với một, hai và ba Leaf. Nút thắt thứ hai là tải ghi UART tại Root: ở mức ba Leaf và chu kỳ 300 ms, toàn mạng ngừng truyền dữ liệu sau 6–26 giây, còn ở chu kỳ 600 ms, ba Leaf chạy liên tục khoảng 2 phút với FPS tổng 4,85 ± 0,96 khung/giây.

Bên cạnh đó, firmware đã được bổ sung watchdog tại Leaf, theo dõi node im lặng, bảo vệ heap tại Root, ghi UART không chặn và gửi lệch pha giữa các Leaf. Firmware cũng đã có chức năng đo độ trễ, đo thời gian tự phục hồi theo luồng dữ liệu,

ghi nhận cây topology và RSSI của từng liên kết. Các chức năng này cung cấp công cụ đo cho các thử nghiệm tiếp theo.

5.1.2 Hạn chế

Trong phạm vi kết quả trình bày ở Chương 4, hệ thống còn một số hạn chế. Hệ thống chưa có cơ chế gửi lại, nên chỉ cần mất một chunk thì toàn bộ khung ảnh bị hủy. Mạng ba Leaf ở chu kỳ 600 ms vẫn hoạt động sát ngưỡng: Chương 4 ghi nhận một đợt tỉ lệ hủy khung tăng lên 7–36 % trong khoảng 20 giây và heap của Root có thời điểm giảm xuống 73 KB.

Về phép đo, thời gian tự phục hồi ghi được phân tán từ 885 đến 32528 ms nên chưa được dùng làm kết quả. Chương 4 chưa trình bày kết quả đo độ trễ và thông lượng theo số hop, cũng như kết quả đo dòng tiêu thụ và thời gian hoạt động bằng pin. Các cải tiến ổn định ngày 30/09 mới có kết quả ban đầu, vì tệp CSV tương ứng không ghi phiên bản firmware.

Về phạm vi thử nghiệm, các node được đặt cố định hoặc cầm tay trên mặt đất, nên các kết quả chưa phản ánh trường hợp node bay hoặc di chuyển liên tục. Các nhận xét ở Chương 4 vì vậy chỉ áp dụng cho cấu hình một Root và tối đa ba Leaf đặt trên mặt đất. Ngoài ra, thư viện painlessMesh truyền bản tin JSON qua TCP và chuyển tiếp theo từng cạnh, nên tốn nhiều thời gian chiếm kênh và không được thiết kế cho nhiều luồng ảnh song song.

5.2 Hướng phát triển

Trên cơ sở các hạn chế đã nêu, hướng phát triển của đề tài được chia thành hai giai đoạn. Giai đoạn ngắn hạn hoàn thiện phép đo và giảm tải cho Root trên nền painlessMesh hiện tại. Giai đoạn dài hạn thay đổi nền tảng truyền thông và mở rộng mô hình theo hướng FANET.

5.2.1 Hướng phát triển ngắn hạn

Hướng thứ nhất là tách việc ghi UART khỏi vòng lặp chính. Khi Root phải ghi UART liên tục, mesh.update() trong loop() bị gọi thưa và toàn mạng ngừng truyền dữ liệu, như đã ghi nhận ở Chương 4. Đề tài đề xuất chuyển việc ghi UART sang một tác vụ FreeRTOS riêng, nhận dữ liệu qua hàng đợi. Theo cơ chế ở mục 2.4, tác vụ ở trạng thái Blocked khi chờ hàng đợi không chiếm CPU, nên vòng lặp chứa mesh.update()

không còn bị chặn bởi thao tác ghi UART. Đề xuất này được kiểm chứng bằng cách so sánh chỉ số loop_max_ms và số khung ảnh bị bỏ tại UART trước và sau khi thay đổi, trong cùng kịch bản ba Leaf, sau đó thử đưa chu kỳ chụp về gần 300 ms.

Hướng thứ hai là chuẩn hóa phương pháp đo thời gian tự phục hồi. Các giá trị ghi được ngày 29/09 phân tán từ 1243 đến 19919 ms, nên giá trị trung bình không có ý nghĩa. Đề tài đề xuất đo lặp từ 5 đến 10 lần và tách riêng hai kịch bản: đổi tuyến qua node trung gian, và mất tín hiệu rồi kết nối lại. Lý do là hai kịch bản này kích hoạt hai cơ chế phát hiện khác nhau, trong đó trường hợp node mất nguồn đột ngột phụ thuộc vào thời gian chờ NODE_TIMEOUT của painlessMesh, nên gộp chung sẽ làm giá trị trung bình không đại diện cho kịch bản nào. Kết quả được kiểm chứng khi mỗi kịch bản có giá trị nhỏ nhất, lớn nhất và trung bình với độ phân tán đủ nhỏ để so sánh.

Hướng thứ ba là bổ sung các phép đo còn thiếu so với nhiệm vụ đề tài. Chương 4 chưa có kết quả theo số hop, trong khi mức suy giảm thông lượng theo số hop là một tiêu chí đánh giá chính ở mục 1.2. Đề tài đề xuất bố trí các node thành chuỗi Root – node chuyển tiếp – Leaf và đo thông lượng, độ trễ ở một, hai và ba hop với cùng chu kỳ chụp, đồng thời đo dòng tiêu thụ của Leaf để ước lượng thời gian chạy pin. Cây topology và số hop do Root gửi lên dashboard cho phép kiểm tra số hop thực tế trong mỗi phép đo.

Hướng thứ tư là bổ sung cơ chế gửi lại có chọn lọc. Hiện nay, chỉ cần mất một chunk thì cả khung ảnh bị hủy. Đề tài đề xuất cơ chế NACK (Negative Acknowledgement) gộp: sau mỗi khung, Root gửi một bản tin liệt kê các chunk còn thiếu để Leaf chỉ gửi lại các chunk đó. Cơ chế này chỉ phát sinh một bản tin xác nhận cho mỗi khung nên tránh được hiện tượng bão bản tin xác nhận; hiệu quả được kiểm chứng bằng cách so sánh tỉ lệ hủy khung ở cùng số Leaf và cùng chu kỳ chụp.

5.2.2 Hướng phát triển dài hạn

Về lâu dài, đề tài có thể phát triển tiếp phiên bản sử dụng API ESP-WIFI-MESH chính thức trên ESP-IDF, hiện đã được xây dựng một phần. Vấn đề của painlessMesh là bản tin phải mã hóa Base64, làm tăng khoảng 33 % kích thước dữ liệu, và được truyền dưới dạng JSON qua TCP theo từng cạnh. ESP-WIFI-MESH truyền dữ liệu nhị phân trực tiếp, dữ liệu gửi lên Root tự đi theo cây và có sẵn cơ chế điều khiển luồng chống tràn bộ đệm, như trình bày ở mục 2.3. Hai phiên bản có thể được so sánh bằng FPS tổng và tỉ lệ hủy khung trong cùng kịch bản ba Leaf.

Một hướng khác là mở rộng mô hình theo các yêu cầu của FANET. painlessMesh tự tổ chức thành cây và không dùng thông tin vị trí, trong khi Bekmezci và cộng sự (2013) dẫn một nghiên cứu mô phỏng, trong đó giao thức định tuyến dựa trên vị trí GPSR cho kết quả tốt hơn các giao thức chủ động và phản ứng trong FANET. Các tác giả cũng chỉ ra rằng phần lớn nghiên cứu giả định topology hai chiều, trong khi FANET thực tế hoạt động trong không gian ba chiều. Vì vậy, đề tài có thể tích hợp module định vị GPS hoặc UWB (Ultra-Wideband) để thử nghiệm định tuyến theo vị trí, đồng thời đặt các node ở nhiều độ cao khác nhau để đánh giá ảnh hưởng của topology ba chiều lên thông lượng và thời gian tự phục hồi.

ESP-NOW (Espressif Systems, n.d.-a) là một hướng nữa để tăng tốc độ truyền ảnh vì giao thức này không cần kết nối TCP giữa các node. Tuy nhiên, ESP-NOW chỉ truyền trực tiếp giữa hai node, nên cần tự xây dựng cơ chế chuyển tiếp để đáp ứng yêu cầu truyền đa chặng. Hướng này chỉ nên được thực hiện sau khi đã có số liệu theo số hop của phiên bản hiện tại để làm mốc so sánh.

TÀI LIỆU THAM KHẢO

Tiếng nước ngoài

Akyildiz, I. F., Wang, X., & Wang, W. (2005). Wireless mesh networks: A survey. *Computer Networks*, *47*(4), 445–487. https://doi.org/10.1016/j.comnet.2004.12.001.

Alteriom (n.d.). painlessMesh (bản fork trên GitHub). Truy cập tháng 9/2026, từ https://github.com/Alteriom/painlessMesh.

Bekmezci, İ., Sahingoz, O. K., & Temel, Ş. (2013). Flying ad-hoc networks (FANETs): A survey. *Ad Hoc Networks*, *11*(3), 1254–1270. https://doi.org/10.1016/j.adhoc.2012.12.004.

Bianchi, G. (2000). Performance analysis of the IEEE 802.11 distributed coordination function. *IEEE Journal on Selected Areas in Communications*, *18*(3), 535–547. https://doi.org/10.1109/49.840210.

Clausen, T., & Jacquet, P. (2003). Optimized link state routing protocol (OLSR). RFC 3626, Internet Engineering Task Force. https://doi.org/10.17487/RFC3626.

Espressif Systems (2024). Arduino release v2.0.17 based on ESP-IDF v4.4.7. Truy cập ngày 09/10/2026, từ https://github.com/espressif/arduino-esp32/releases.

Espressif Systems (n.d.-a). ESP-NOW. ESP-IDF programming guide. Truy cập tháng 9/2026, từ https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html.

Espressif Systems (n.d.-b). ESP-WIFI-MESH. ESP-IDF programming guide. Truy cập tháng 9/2026, từ https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/esp-wifi-mesh.html.

Espressif Systems (n.d.-c). ESP-WIFI-MESH overview. Truy cập ngày 07/10/2026, từ https://www.espressif.com/en/products/sdks/esp-wifi-mesh/overview.

Espressif Systems (n.d.-d). ESP32 series datasheet, version 5.3. Truy cập ngày 07/10/2026, từ https://documentation.espressif.com/esp32_datasheet_en.pdf.

Espressif Systems (n.d.-e). FreeRTOS (IDF). ESP-IDF programming guide. Truy cập ngày 09/10/2026, từ https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos_idf.html.

Espressif Systems (n.d.-f). Watchdogs. ESP-IDF programming guide. Truy cập ngày 09/10/2026, từ https://docs.espressif.com/projects/esp-idf/en/latest/api-reference/system/wdts.html.

Open Mesh (n.d.). B.A.T.M.A.N. advanced (batman-adv). Truy cập tháng 9/2026, từ https://www.open-mesh.org/projects/batman-adv.

painlessMesh (n.d.). painlessMesh library. Truy cập tháng 9/2026, từ https://gitlab.com/painlessMesh/painlessMesh.

Perkins, C., Belding-Royer, E., & Das, S. (2003). Ad hoc on-demand distance vector (AODV) routing. RFC 3561, Internet Engineering Task Force. https://doi.org/10.17487/RFC3561.

Perkins, C. E., & Bhagwat, P. (1994). Highly dynamic destination-sequenced distance-vector routing (DSDV) for mobile computers. *ACM SIGCOMM Computer Communication Review*, *24*(4), 234–244.

PHỤ LỤC A. MÃ NGUỒN VÀ TÀI NGUYÊN DỰ ÁN

Toàn bộ mã nguồn của đề tài được lưu trên GitHub tại `https://github.com/JoteraJ/do-an-chuyen-nganh-meshdrone`. Thư mục thuattoan/ chứa firmware node Root (node_root/), firmware node Leaf (node_camera/), các tệp header dùng chung (shared/: chunk_protocol.h, crc16.h, base64_util.h), middleware Node.js kèm Web Dashboard (middleware/) và đặc tả giao thức UART (UART_PROTOCOL.md). Thư mục esp_idf_migration/ chứa phiên bản thử nghiệm dùng ESP-WIFI-MESH trên ESP-IDF, đã dừng phát triển.

Các tệp kiểm thử gồm mo_phong_reassembly.py (mô phỏng thuật toán ghép ảnh, đạt 5/5) và test_protocol.js (kiểm thử bộ tách khung UART, đạt 7/7). Bộ tách khung được viết tách riêng khỏi phần đọc cổng serial và Socket.IO, nên có thể kiểm thử bằng phần mềm mà không cần board thật. Số liệu đo được lưu dưới dạng tệp CSV xuất từ tab So sánh của Web Dashboard.